2026-2 Midterm

Database Systems · Midterm Study Notes

6 units · 29 lessons · same content as the trainer's Learn tab (English)

U1 Database System Concepts and Architecture

L1-1 · DB, DBMS and Database System

L1-2 · Four Characteristics of the Database Approach

L1-3 · Data Models, Three-Schema Architecture, Data Independence

U2 The ER Model (Conceptual Design)

L2-1 · Design Process, Entities and Attributes

L2-2 · Relationship Types, Degree, Roles, Constraints

L2-3 · Relationship Attributes, Weak Entities

L2-4 · Refining the COMPANY Design, Practice (MOVIE, CONF_REVIEW)

U3 The Relational Data Model and ER-to-Relational Mapping

L3-1 · Basic Concepts of the Relational Model

L3-2 · Keys and Integrity Constraints

L3-3 · Update Operations and Constraint Violations

L3-4 · ER-to-Relational Mapping ①: Entities, Multivalued Attributes, Weak Entities

L3-5 · ER-to-Relational Mapping ②: Relationship Types

U4 Basic SQL (DDL, DML, SELECT)

L4-1 · Introduction to SQL, CREATE SCHEMA/TABLE, Constraints

L4-2 · Referential Triggered Actions, DROP, ALTER

L4-3 · INSERT, DELETE, UPDATE

L4-4 · SELECT-FROM-WHERE, Aliases

L4-5 · Multisets, Set Operations, ORDER BY, Operators

U5 Advanced SQL (Nested Queries, Joins, Aggregation, Views)

L5-1 · Nested Queries, IN, ALL/ANY

L5-2 · EXISTS / NOT EXISTS, Division Queries

L5-3 · Joined Tables, NATURAL, INNER/OUTER

L5-4 · Aggregate Functions, GROUP BY, HAVING

L5-5 · Assertions, Triggers

L5-6 · Views

U6 Relational Algebra

L6-1 · Introduction to Relational Algebra, Query Trees

L6-2 · Unary Operations: SELECT, PROJECT, RENAME

L6-3 · Set Operations, Cartesian Product

L6-4 · JOIN, DIVISION, Complete Set

L6-5 · Additional Operations: Aggregation, Grouping, Outer Joins

L6-6 · Relational Algebra Writing Practice

U1 · 1_course_intro.pdf + 2_intro-ER.pdf p.3–10

Database System Concepts and Architecture

The differences between a DB, a DBMS and a database system, the four characteristics of the database approach, the three-schema architecture and data independence. This part is ideal for one-line definition questions, so memorize the English keywords along with each idea.

DB, DBMS and Database System

Slides: 1_course_intro p.15–18, 24

§ DB, DBMS, Database System

Term	Definition
Database (DB)	A logically coherent collection of related data. It is managed by a DBMS
DBMS	Software that efficiently "manages" data in a structured way. It is used for **defining, constructing, manipulating and sharing** databases. e.g., IBM DB2, Oracle, MS SQL Server, MySQL, SQLite
Database System	**DB + DBMS software**

> Exam point: pair the four DBMS verbs (define, construct, manipulate, share) with "database system = DB + DBMS".

§ Why use a DBMS?

Slide example: storing a messaging app's data (friend info, messages) in a **text file**, one record per line with attributes separated by commas, causes three groups of problems.

- **Data Integrity**: How do we enforce the message length limit? What if a friend's info is entered twice by mistake? What about friends with the same name?
- **Data Management**: How do we find a specific message quickly? What if many users read and write at the same time?
- **Data Durability**: What if the machine crashes while the app is writing? How do we recover after a crash?

→ We cannot solve these technical problems ourselves every time, so we use a DBMS. Applications: e-commerce, banking, social media, universities/libraries, airline and railway reservations.

§ When NOT to use a DBMS

Overhead of a DBMS

- **High initial investment** in hardware, software and training
- The cost of **generality** that a DBMS provides for defining and processing data
- Overhead of security, concurrency control, recovery and integrity functions → "too heavy"

When a customized database is better

- Simple, well-defined applications that are **not expected to change at all** (no insertions, deletions or updates)
- **Stringent real-time requirements** that a DBMS cannot meet
- **Embedded systems** with limited storage capacity
- **No multiple-user access** to the data

Four Characteristics of the Database Approach

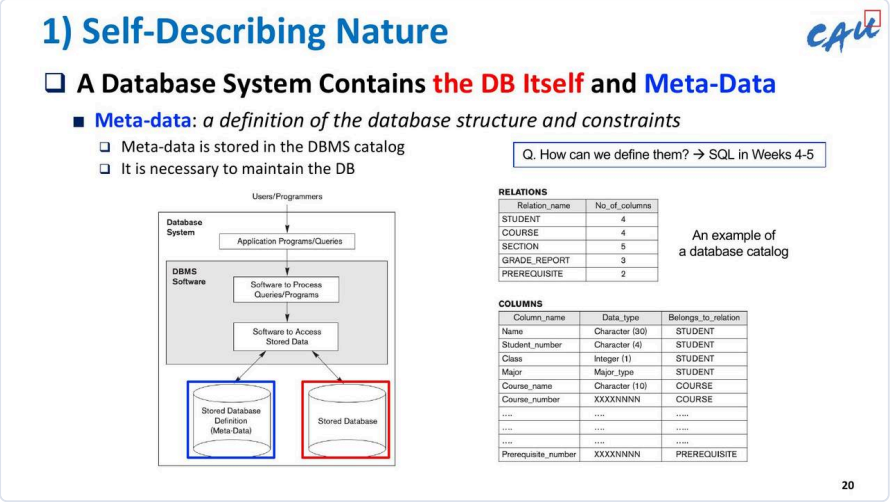
Slides: 1_course_intro p.19–25

§ 1) Self-Describing Nature

A database system contains not only the database itself but also **metadata**.

- **Metadata**: the definition of the database structure and constraints
- Metadata is stored in the **DBMS catalog**. It is essential for maintaining the database
- How to define it = SQL DDL (weeks 4–5)

Trap: metadata is stored in the "catalog". In a file system, the data structure definitions are embedded in the application programs.



01 p.20 · Database system = stored database + metadata (catalog). Catalog examples: RELATIONS, COLUMNS tables

§ 2) Data Abstraction

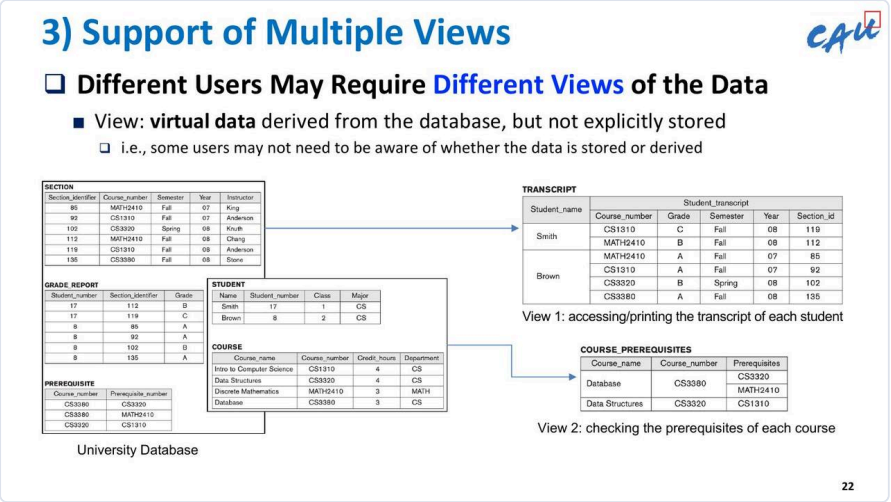
Insulation between programs and data

- A database system provides a **conceptual representation (logical level)** of data: objects, their properties and their interrelationships
- It hides the complex **physical-level** details (how data is stored, how operations are implemented)
- Programs are not affected when the data structure changes → **program-data independence**

§ 3) Support of Multiple Views

Each user may need a different perspective of the data.

- **View**: **virtual data that is derived from the database but not explicitly stored**
- Users do not need to know whether the data they see is stored or derived
- e.g., in a university DB, view 1 = print each student's transcript, view 2 = check each course's prerequisites

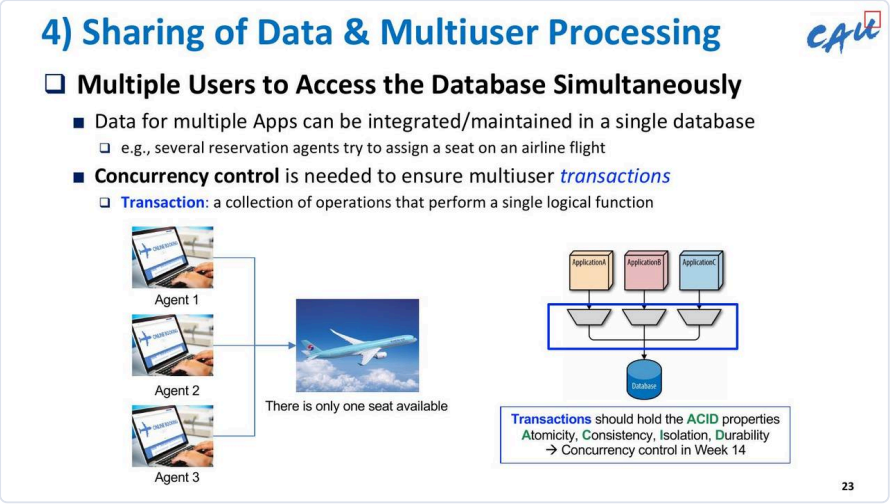


01 p.22 · Two views of the university DB: each student's transcript (TRANSCRIPT), each course's prerequisites (COURSE_PREREQUISITES)

§ 4) Sharing of Data and Multiuser Transaction Processing

- Multiple users access the database **at the same time**. Data from many applications is integrated and managed in a single database
- e.g., several reservation agents try to assign the last remaining seat on a flight at the same time
- **Concurrency control** is required
- **Transaction**: a collection of operations that performs a single logical function
- A transaction must satisfy **ACID**: **A**tomicity, **C**onsistency, **I**solation, **D**urability → week 14

> Exam point: know the four characteristics' exact English names + one line each: "self-describing / data abstraction / multiple views / sharing & multiuser transaction processing".



01 p.23 · Three reservation agents competing for one seat → concurrency control, ACID properties of transactions

Data Models, Three-Schema Architecture, Data Independence

Slides: 2_intro-ER p.3–10

§ Three categories of data models

Data model: a collection of concepts that describe the structure of a database. A "tool" for data abstraction (data types, relationships, constraints, retrieval and update operations).

- Abstraction = emphasizing the essential features needed to understand the data while hiding the details of how data is organized and stored

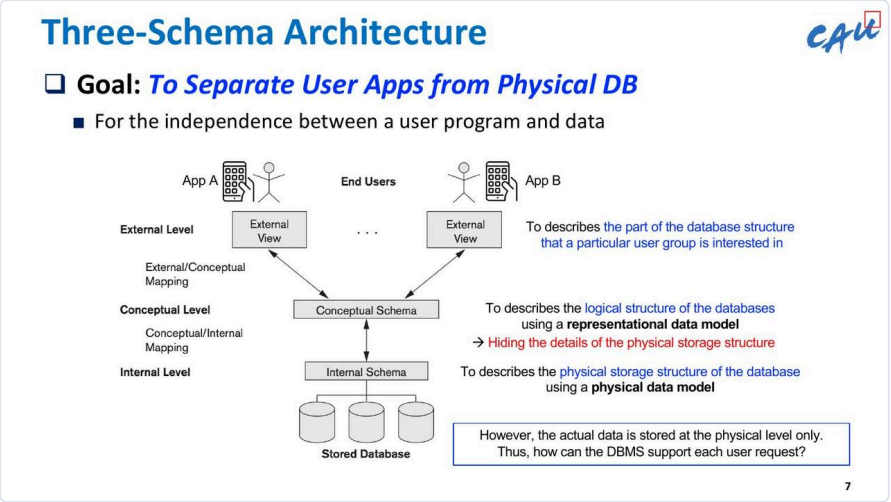
Category	Level	Description
Conceptual	high-level	Concepts close to the way users perceive data. e.g., the ER model (EMPLOYEE, DEPARTMENT, PROJECT)
Representational	intermediate	How the DBMS manages data. Hides physical storage details but can be implemented directly on a computer. e.g., the relational data model
Physical	low-level	How data is stored in the computer: record formats, record orderings, access paths. Meant for specialists, not typical end users

§ Three-Schema Architecture

Goal: separate user applications from the physical database → independence between user programs and data.

Level	Schema	What it describes
External	External schema / user view	The part of the database that a particular user group is interested in
Conceptual	Conceptual schema	The logical structure of the whole database, described with a **representational data model**: entities, data types, relationships, user operations, constraints. Hides physical storage structures
Internal	Internal schema	The physical storage structure of the database, described with a **physical data model**: details of data storage and access paths

Trap: actual data is stored **only at the physical (internal) level**. The external and conceptual schemas are only "descriptions".



02 p.7 · Three-schema architecture (external, conceptual, internal levels and mappings)

§ Mappings between schemas

Since actual data exists only at the internal level, the DBMS must **transform** requests and results between levels.

- Request: transformed external schema → conceptual schema → internal schema, then processed over the stored database
- Result: data extracted from the stored database is reformatted to match the user's external view
- **Mapping information is also stored in the catalog**

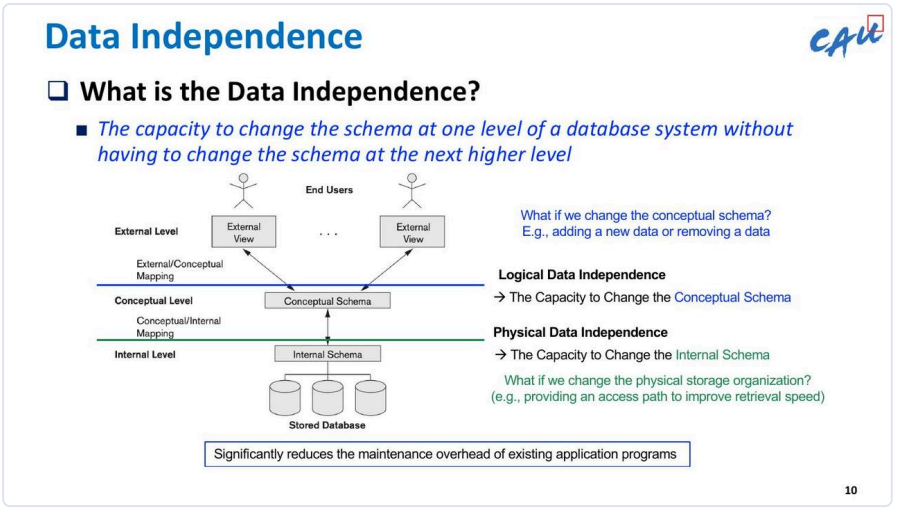
§ Data Independence

The capacity to change the schema at one level without having to change the schema at the next higher level.

Type	What changes	Example
Logical data independence	**Conceptual schema**	Adding new data, removing data → external schemas and application programs stay the same
Physical data independence	**Internal schema**	Reorganizing storage structures, adding an access path (index) to speed up retrieval → conceptual schema stays the same

→ It **greatly reduces the maintenance burden** on existing application programs.

> Exam point: do not mix them up: "logical = change to the conceptual schema, physical = change to the internal schema". Physical independence is easier to achieve (adding an index is common).



02 p.10 · Logical data independence (change to the conceptual schema) vs physical data independence (change to the internal schema)

U2 · 2_intro-ER.pdf p.11–47 + 2-2_ER_model-practices.pdf

The ER Model (Conceptual Design)

Requirements → entities and attributes → relationships and constraints → refinement. The exam is likely to give requirement sentences and ask you to draw an ER diagram or write (min, max), so practice turning "a requirement sentence → the right notation".

Design Process, Entities and Attributes

Slides: 2_intro-ER p.12–21

§ The database design process

1. **Requirements Collection and Analysis**: understand the data requirements of prospective users. Output = a detailed set of user requirements
2. **Conceptual Design**: choose a data model and translate the requirements into a **conceptual schema** (ER model)
3. **Implementation**: implement the database from the conceptual schema
   - **Logical design**: the database schema (e.g., relational schema; ER-to-relational mapping in week 3)
   - **Physical design**: the physical layout of the database

COMPANY requirements: the company is organized into **departments** (unique name and number, several locations, a manager and the manager's start date); a department controls a number of **projects** (unique name and number, a single location). An **employee** has a name, SSN, address, salary, sex, birth date and the projects they work on; for insurance purposes, each employee's **dependents** (name, sex, birth date, relationship) are also kept.

§ Entities and attributes

ER (Entity-Relationship) model: describes data as **collections of entities and relationships**. Drawn as an ER diagram.

- **Entity**: a "thing" in the real world with an **independent existence**, distinguishable from other things. It may be a physical object (an employee) or a conceptual one (a company)
- **Attribute**: a particular property that describes an entity. e.g., employee: name, address, age, home phone
- **Relationship**: an association among several entities

§ Types of attributes

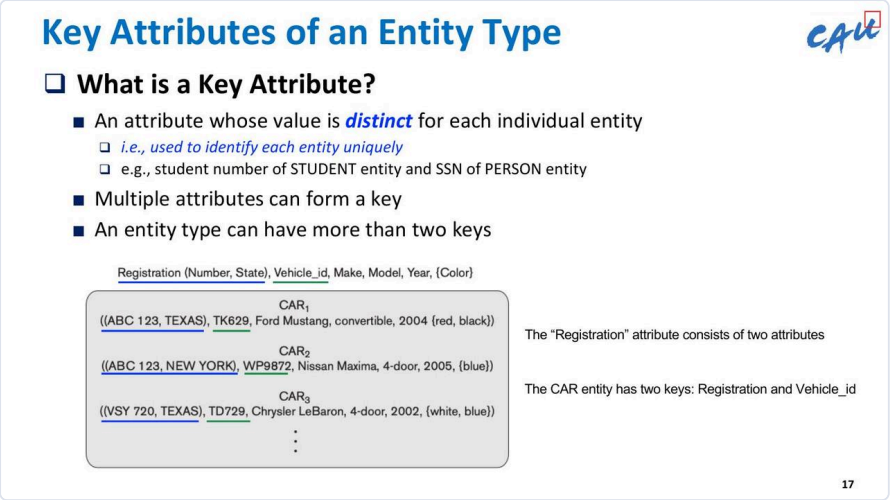
Classification	Pair	Examples
① Composite vs simple	**Composite**: can be divided into smaller parts / **Simple (atomic)**: not divisible	Address = (Street_address, City, State, Zip) / City, Zip
② Single-valued vs multivalued	**Single-valued**: one value per entity / **Multivalued**: a set of values	Age, SSN / Phone numbers, College degrees, a department's Locations
③ Stored vs derived	**Stored**: actually stored in the DB / **Derived**: computed from other attributes	Birth_date / Age (derived from birth date)

Trap: do not confuse "multivalued" with "composite". Composite = one value splits into several parts; multivalued = there are several values. An attribute can be both (e.g., {Address_phone(Phone, Address)}).

§ Key attributes

Key attribute: an attribute whose values are **distinct** for each entity → uniquely identifies each entity.

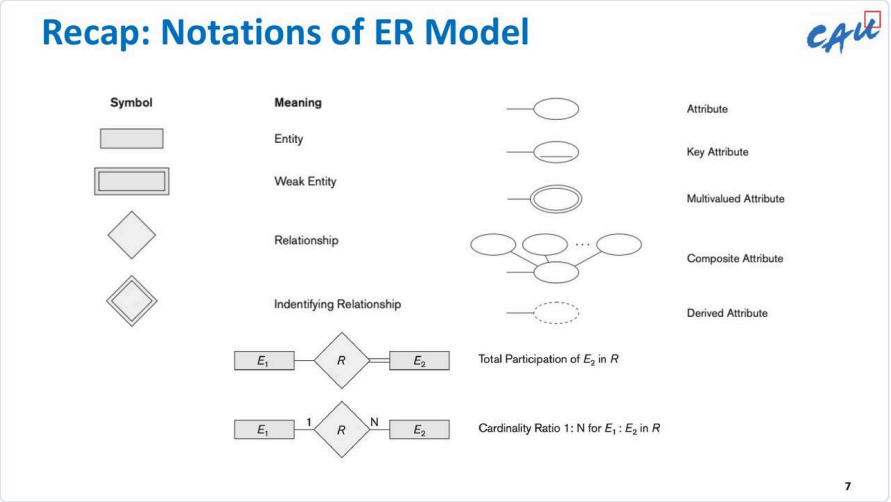
- e.g., a STUDENT's student ID, a PERSON's SSN
- Several attributes together can form one key (e.g., CAR's Registration = (Number, State), a composite key)
- An entity type can have **more than one** key (e.g., CAR's keys = Registration, Vehicle_id)
- Shown **underlined** in the diagram



02 p.17 · Two keys of CAR: composite key Registration(Number, State) and Vehicle_id

§ ER diagram notation

Symbol	Meaning
Rectangle	Entity type
Double rectangle	Weak entity type
Diamond	Relationship type
Double diamond	Identifying relationship type
Oval	Attribute
Oval with underlined name	Key attribute
Double oval	Multivalued attribute
Oval with ovals attached	Composite attribute
Dashed oval	Derived attribute
E_1 — R ═ E_2 (double line)	**Total participation** of E_2 in R
E_1 —1— R —N— E_2	Cardinality ratio 1:N
R —(min,max)— E	Structural constraint (min, max) on participation of E in R



02-2 p.7 · Summary of ER notation (weak entity, identifying relationship, multivalued/composite/derived attributes, total participation, 1:N)

§ Initial conceptual design: COMPANY

- **DEPARTMENT**: Name, Number, {Locations}, Manager, Manager_start_date. Name and Number are each keys (both unique). Locations is multivalued
- **PROJECT**: Name, Number, Location, Controlling_department. Name and Number are keys
- **EMPLOYEE**: Name (composite: Fname, Minit, Lname), SSN (key), Address, Salary, Birth_date, Sex, Department, Supervisor, {Works_on(Project, Hours)}
- **DEPENDENT**: Employee, Dependent_name, Sex, Birth_date, Relationship

Problem: attributes such as Manager, Department, Supervisor, Works_on and Employee actually **refer to other entities**. Relationships are hidden **implicitly** as attributes → they must be turned into explicit **relationship types** (L2-4 refinement).

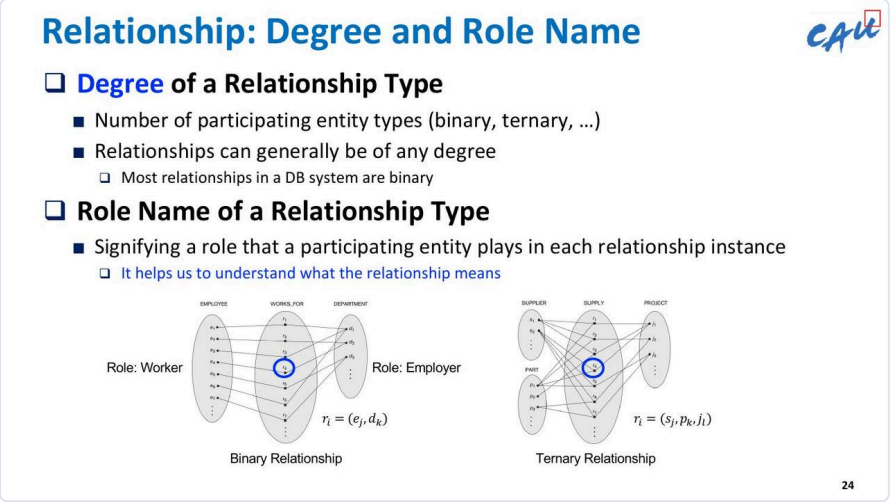
Relationship Types, Degree, Roles, Constraints

Slides: 2_intro-ER p.23–30, 36–37

§ Relationship type, degree, role names

Relationship type: described by a name + the participating entity types. It defines a set of associations among the participating entities.

- e.g., WORKS_FOR: between EMPLOYEE and DEPARTMENT. It associates each employee with the department they work for
- **Degree**: the number of participating entity types. Binary, ternary, ... most are binary
  - Binary example: $r_i = (e_j, d_k)$; ternary example: SUPPLY's $r_i = (s_j, p_k, j_l)$ (SUPPLIER, PART, PROJECT)
- **Role name**: the role a participating entity plays in each relationship instance. Helps explain what the relationship means. e.g., in WORKS_FOR, EMPLOYEE = worker, DEPARTMENT = employer

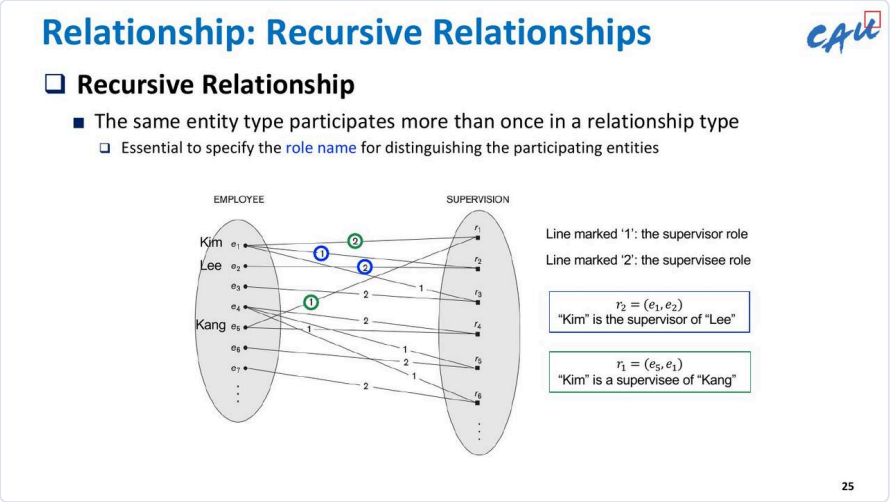


02 p.24 · Binary relationship WORKS_FOR (roles worker/employer) and ternary relationship SUPPLY

§ Recursive relationship

A relationship in which **the same entity type participates more than once in different roles**. **Role names are required** to distinguish the participants.

- e.g., SUPERVISION: EMPLOYEE participates as supervisor (line '1') and as supervisee (line '2')
- $r_2 = (e_1, e_2)$: Kim is Lee's supervisor / $r_1 = (e_5, e_1)$: Kim is Kang's supervisee



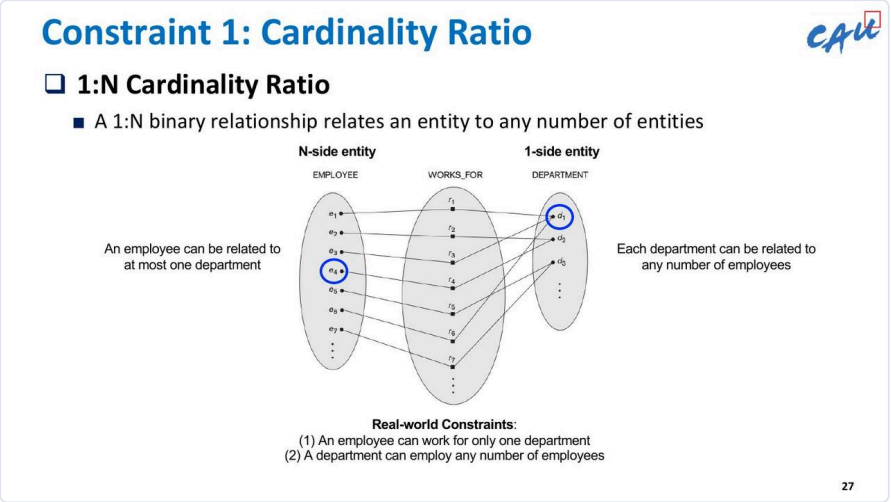
02 p.25 · Recursive relationship SUPERVISION: line 1 = supervisor, line 2 = supervisee

§ Constraint ① Cardinality ratio

The maximum number of relationship instances that an entity can participate in. For binary relationships: 1:1, 1:N, M:N.

Ratio	Example	Real-world constraint
1:N	WORKS_FOR (DEPARTMENT 1 : EMPLOYEE N)	An employee works for only one department; a department can have any number of employees
1:1	MANAGES (EMPLOYEE 1 : DEPARTMENT 1)	An employee manages at most one department; a department has at most one manager
M:N	WORKS_ON (EMPLOYEE M : PROJECT N)	An employee works on many projects; a project has many employees

Trap: in 1:N, one entity on the **N-side** is related to **at most one** entity on the 1-side. "One employee → at most one department", so EMPLOYEE is the N-side.

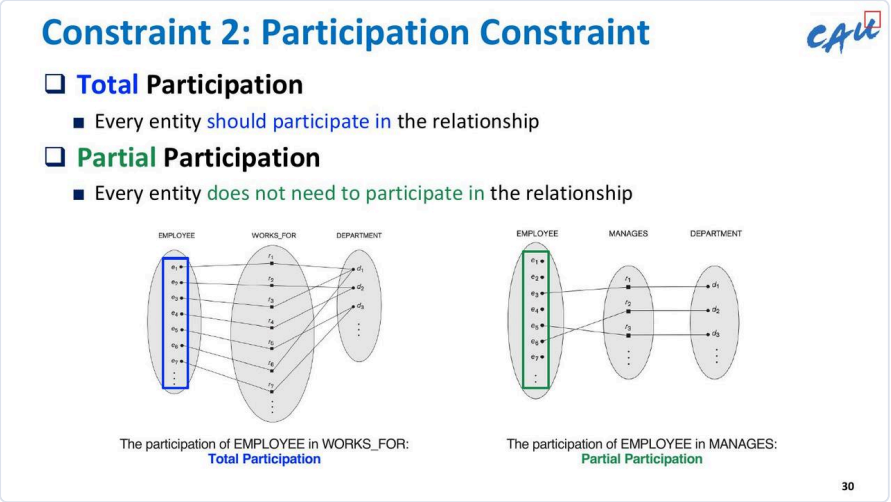


02 p.27 · 1:N relationship WORKS_FOR: EMPLOYEE on the N-side, DEPARTMENT on the 1-side

§ Constraint ② Participation constraint

Specifies **whether the existence of an entity depends on its being related to another entity**. It concerns the minimum number of participations.

- **Total participation**: **every** entity must participate in the relationship. Double line. e.g., every employee must work for some department, so EMPLOYEE has total participation in WORKS_FOR (= existence dependency)
- **Partial participation**: not every entity needs to participate. Single line. e.g., not every employee manages a department, so EMPLOYEE has partial participation in MANAGES



02 p.30 · EMPLOYEE has total participation in WORKS_FOR and partial participation in MANAGES

---

§ (min, max) notation

Attach a **(min, max)** pair to each participation to show cardinality and participation together. 0 ≤ min ≤ max, max ≥ 1.

- **min = 0 → partial participation, min ≥ 1 → total participation**
- max = the maximum number of relationship instances that the entity can participate in
- e.g., MANAGES: EMPLOYEE (0,1), DEPARTMENT (1,1)

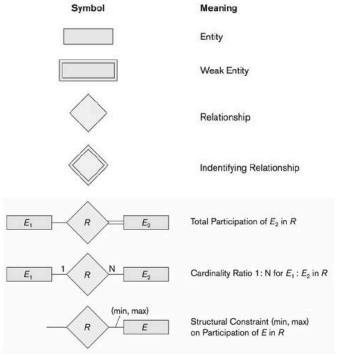
Trap: (min,max) is written **on that entity's side and means how many times one such entity** participates in the relationship. The "1" and "N" of 1:N notation are written on the **opposite** side (DEPARTMENT —1— WORKS_FOR —N— EMPLOYEE). So for WORKS_FOR the (min,max) values look reversed: EMPLOYEE (1,1), DEPARTMENT (4,N).

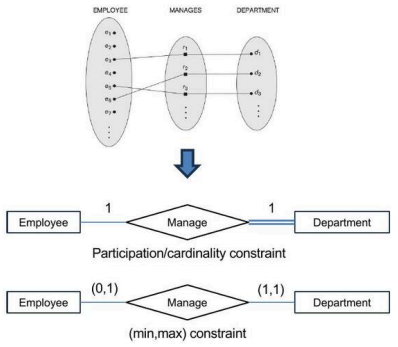
Slide Exercise 1 (INSTRUCTOR–TEACHES–COURSE–USES–TEXT):

- A course may or may not use a textbook, but not more than five → COURSE's participation in USES **(0, 5)**
- A course is taught by at least 1 and at most 4 instructors → COURSE's participation in TEACHES **(1, 4)**
- A textbook is used in some courses → TEXT's participation in USES **(1, N)**
- An instructor teaches 2 to 4 courses → INSTRUCTOR's participation in TEACHES **(2, 4)**

Representation of Relationship Types

❑ **How to Represent a Relationship Type and its Constraints?**





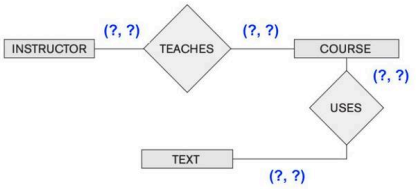
36

02 p.36 · Relationship notation and (min,max): in MANAGES, EMPLOYEE (0,1), DEPARTMENT (1,1)

Exercise 1: Constraints on ER Diagram

❑ **Suggest the (min, max) constraints on the following diagram**

- A course may or may not use a textbook
- A course may not use more than five books
- A course can be taught by at least one instructor and by a maximum of 4 instructors
- A textbook is used in some courses
- An instructor teaches from two to four courses



37

02 p.37 · Exercise 1: fill in (min,max) for INSTRUCTOR–TEACHES–COURSE–USES–TEXT

Relationship Attributes, Weak Entities

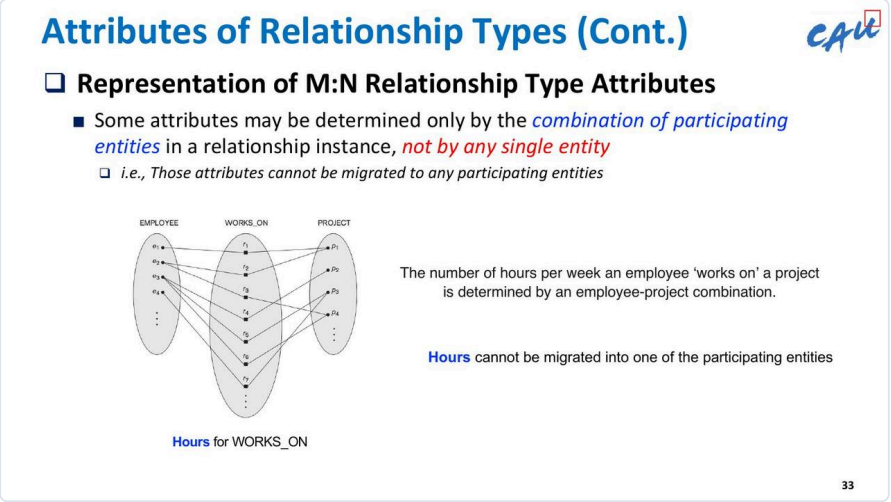
Slides: 2_intro-ER p.31–35

§ Attributes of relationship types

Relationship types can also have attributes. e.g., MANAGES' StartDate, WORKS_ON's Hours, WORKS_FOR's StartDate.

Relationship	Where the attribute can be migrated
1:1	To **either** participating entity type (MANAGES.StartDate → DEPARTMENT or EMPLOYEE)
1:N	**Only to the entity type on the N-side** (WORKS_FOR.StartDate → EMPLOYEE)
M:N	**Cannot be migrated to either entity**, because it is determined only by the combination of participating entities (WORKS_ON.Hours is determined by the employee–project combination)

Exam point: "Why can a 1:N relationship attribute go only to the N-side?" → one N-side entity participates in at most one relationship instance, so its value is uniquely determined. A 1-side entity participates in many instances, so it would have many values.



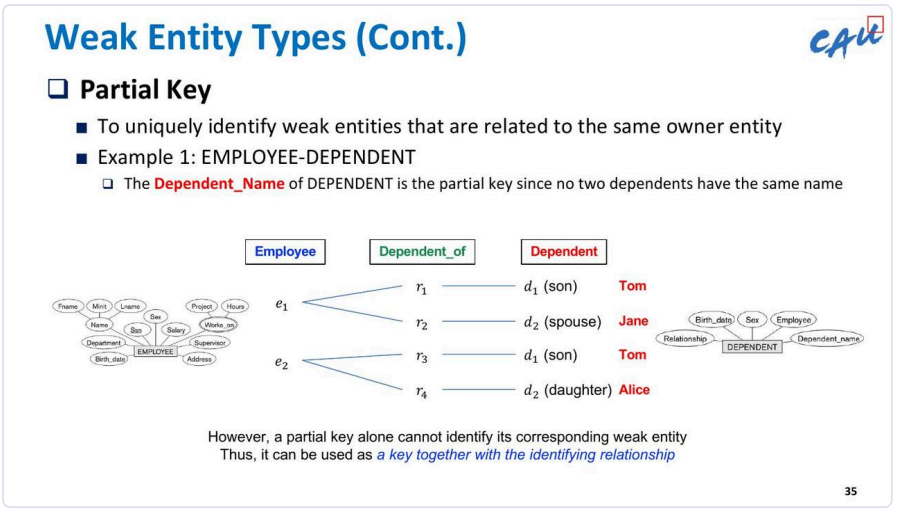
02 p.33 · Hours of the M:N relationship WORKS_ON: determined only by the employee–project combination

§ Weak entity types

Weak entity type: an entity type that **does not have key attributes of its own**. It depends on an **identifying (owner) entity type**.

- A weak entity must be related to an identifying entity
- **Identifying relationship**: the relationship that relates a weak entity type to its owner entity type. Double diamond
- A weak entity type **always has total participation** in its identifying relationship
- **Partial key**: the attribute that distinguishes weak entities related to the same owner. Dashed underline
  - e.g., DEPENDENT's Dependent_name (assuming no two dependents of the same employee share a name)

- The partial key alone cannot identify an entity (another employee may also have a dependent named Tom) → identified by **owner's key + partial key**



02 p.35 · Partial key Dependent_name of the weak entity DEPENDENT (Tom belongs to two employees)

Refining the COMPANY Design, Practice (MOVIE, CONF_REVIEW)

Slides: 2_intro-ER p.38–47, 2-2 p.14–26

§ Refining the COMPANY design

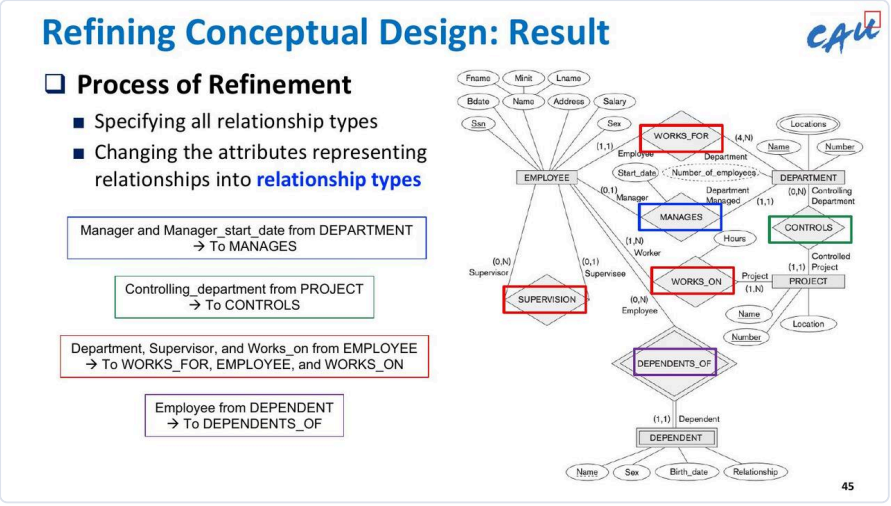
Change attributes that represent relationships **into relationship types**.

Relationship	Participants	Cardinality	Participation	Attributes
MANAGES	EMPLOYEE, DEPARTMENT	1:1	EMPLOYEE partial, DEPARTMENT total	Start_date (Manager_start_date)
WORKS_FOR	EMPLOYEE, DEPARTMENT	1:N (DEPARTMENT 1, EMPLOYEE N)	Both total	None
CONTROLS	DEPARTMENT, PROJECT	1:N (DEPARTMENT 1, PROJECT N)	PROJECT total, DEPARTMENT partial	None
SUPERVISION	EMPLOYEE (supervisor, supervisee)	1:N (supervisor 1, supervisee N)	Both partial	None
WORKS_ON	EMPLOYEE, PROJECT	M:N	Both total	Hours
DEPENDENTS_OF	EMPLOYEE (identifying), DEPENDENT (weak)	1:N (EMPLOYEE 1)	EMPLOYEE partial, DEPENDENT total	None (identifying relationship)

What moved: DEPARTMENT's Manager and Manager_start_date → MANAGES; PROJECT's Controlling_department → CONTROLS; EMPLOYEE's Department, Supervisor, Works_on → WORKS_FOR, SUPERVISION, WORKS_ON; DEPENDENT's Employee → DEPENDENTS_OF.

Caution: the SUPERVISION description on slide p.42 has the 1-side/N-side roles swapped. One supervisor has many supervisees, so **the supervisor is the 1-side**, and the resulting diagram (p.45) also gives (min,max) supervisor (0,N), supervisee (0,1).

Summary of the 3 steps of ER schema design: (1) describe entity types and attributes from the requirements → (2) specify relationship types and constraints → (3) change attributes that represent relationships into relationship types. Focus on the **schema**, not instances.



02 p.45 · Refined COMPANY ER diagram (six relationships with (min,max))

§ Practice: MOVIE / CONF_REVIEW

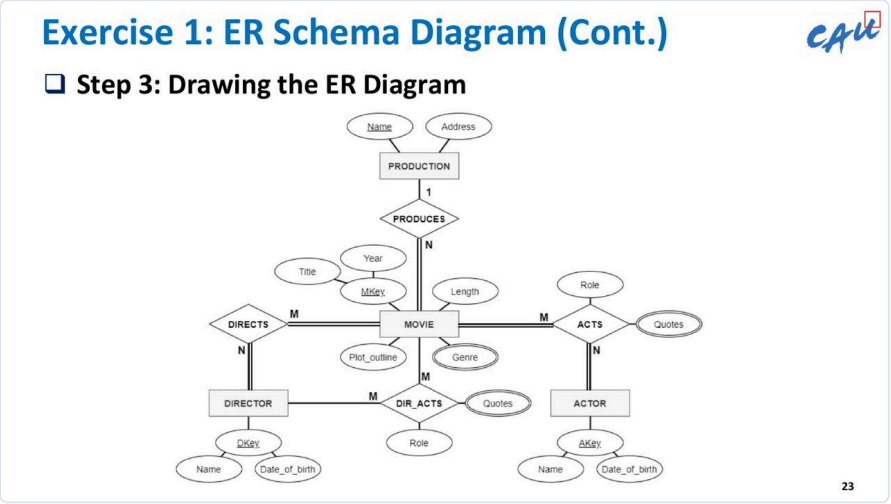
MOVIE DB (2-2 p.14–23)

- MOVIE: composite key MKey(Title, Year), Length, Plot_outline, {Genre} multivalued
- ACTOR, DIRECTOR: key (Name, Date_of_birth). PRODUCTION: key Name, Address
- **PRODUCES** (PRODUCTION 1 : MOVIE N): a movie is produced by one production company. MOVIE total participation
- **ACTS** (ACTOR M : MOVIE N): relationship attributes Role, {Quotes} (multivalued). Total participation on both sides
- **DIRECTS** (DIRECTOR M : MOVIE N): MOVIE total participation (at least one director)
- **DIR_ACTS**: a director may also act in a movie (Role, {Quotes})

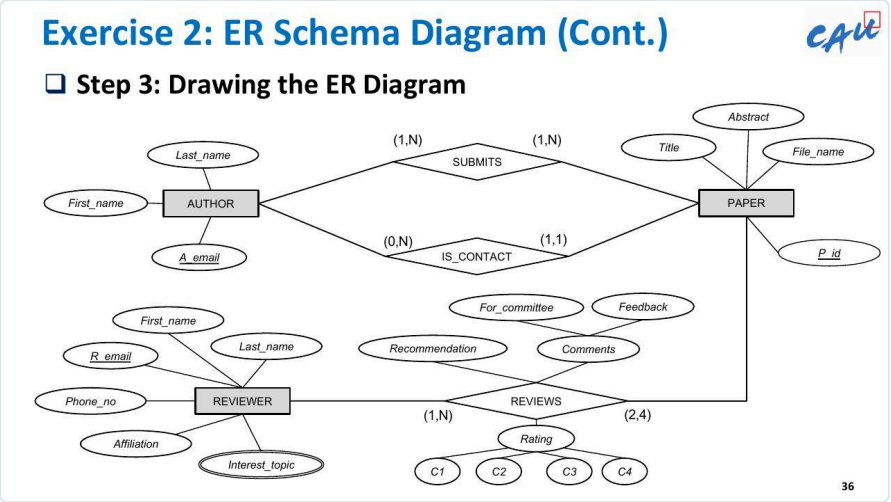
CONF_REVIEW DB (2-2 p.24–26, solution in db-exercises)

- AUTHOR(A_email key, First_name, Last_name), PAPER(P_id key, Title, Abstract, File_name), REVIEWER(R_email key, name, Phone_no, Affiliation, {Interest_topic})
- **SUBMITS** AUTHOR (1,N) – PAPER (1,M): an author is recorded only after submitting a paper; a paper may have multiple authors (M:N)
- **IS_CONTACT** AUTHOR (0,N) – PAPER (1,1): each paper has exactly one contact author. SUBMITS alone cannot express this requirement, so it is a separate relationship
- **REVIEWS** REVIEWER (1,N) – PAPER (2,4): 2 to 4 reviewers per paper. Relationship attributes: Recommendation; Comments(For_committee, Feedback) composite; Rating(C1 Technical_merit, C2 Readability, C3 Originality, C4 Relevance) composite

> Exam point: "exactly one per ..." = (1,1); "may or may not" = min 0; "at least a, at most b" = (a,b); "many" = max N. The key skill is spotting these words in a requirement sentence and turning them into notation.



02-2 p.23 · MOVIE DB ER diagram



db-exercises p.36 · CONF_REVIEW DB ER diagram (SUBMITS, IS_CONTACT, REVIEWS)

U3 · 3-1_RDB.pdf

The Relational Data Model and ER-to-Relational Mapping

Terminology (relation, tuple, attribute, domain, degree), the key family (superkey ⊃ key = candidate key ∋ primary key), the three integrity constraints, which constraints insert/delete/update can violate, and ER-to-relational mapping. For mapping, all you really need is to get "where does the FK go" right.

Basic Concepts of the Relational Model

Slides: 3-1 p.4–12

§ The relational data model

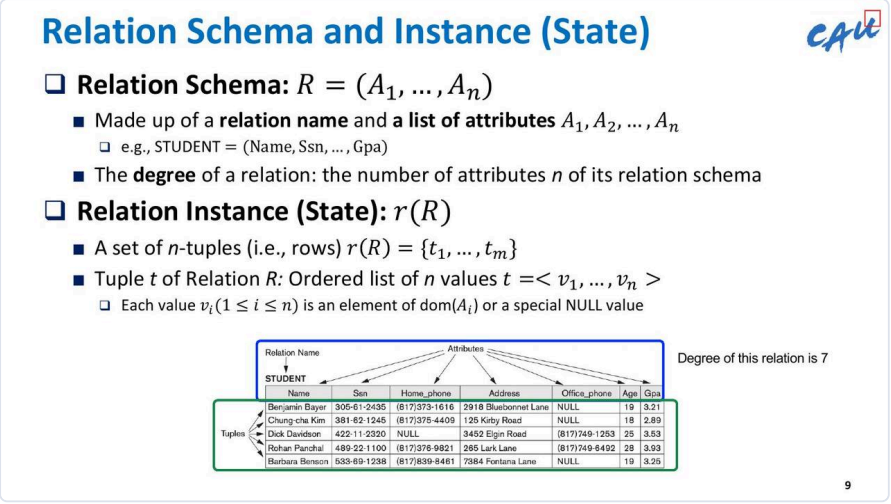
- Proposed by **Ted Codd** of IBM in 1970
- Represents data as **relations (tables)**. Each relation consists of **tuples (rows)** and **attributes (column headers)**
- Each row = a fact about a particular entity (a collection of related data values); attributes specify how to interpret the values in each row

Four advantages: **Simplicity** (flat table structure), **Data integrity** (ensures valid, consistent and accurate data), **Ease of data manipulation** (powerful high-level languages such as SQL handle complex conditions flexibly), **Security** (authentication, authorization, encryption)

§ Domain, relation schema, instance

- **Domain D**: a **set of atomic values** allowed for an attribute. Domain = data type + constraints
  - e.g., Usa_phone_numbers: format (ddd)ddd-dddd; Employee_ages: integers between 15 and 80
- **Relation schema R(A_1, ..., A_n)**: relation name + list of attributes. e.g., STUDENT(Name, Ssn, ..., Gpa)
- **Degree**: the **number of attributes n** of a relation schema. e.g., STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa) has degree 7
- **Relation instance (state) r(R)**: a **set** of n-tuples {t_1, ..., t_m}
- **Tuple t**: an **ordered** list of n values <v_1, ..., v_n>. Each v_i is an element of dom(A_i) or the special value **NULL**

Trap: degree = number of attributes (columns). The number of tuples (rows) is called the cardinality. Do not mix them up.



03 p.9 · Relation name, attributes, tuples, degree (7)

§ Characteristics of relations

- **Values in a tuple must be atomic** (not further divisible)
  - **Composite and multivalued attributes are not allowed**

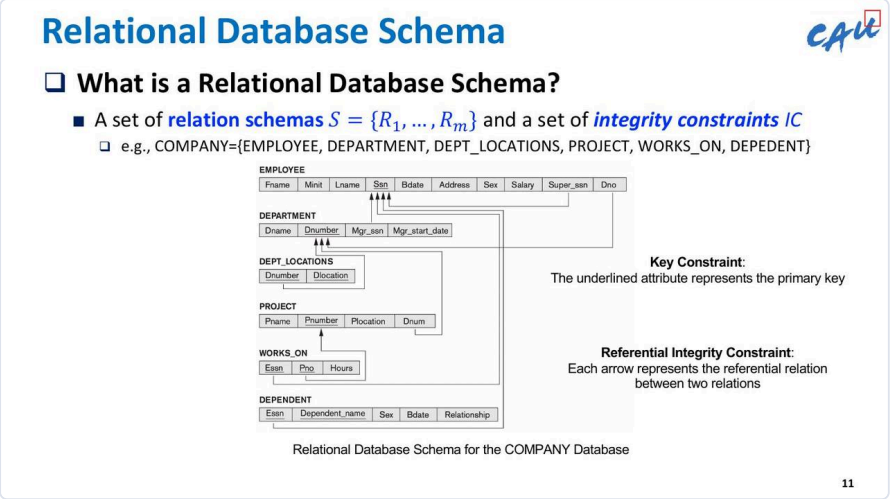
- Multivalued attribute → represented by a **separate relation**
- Composite attribute → represented only by its **simple component attributes**

- **NULL**: used when a value is unknown, unavailable, or does not apply to that tuple (undefined/not applicable)
- A relation is a **set** of tuples, so by definition **all tuples must be distinct**

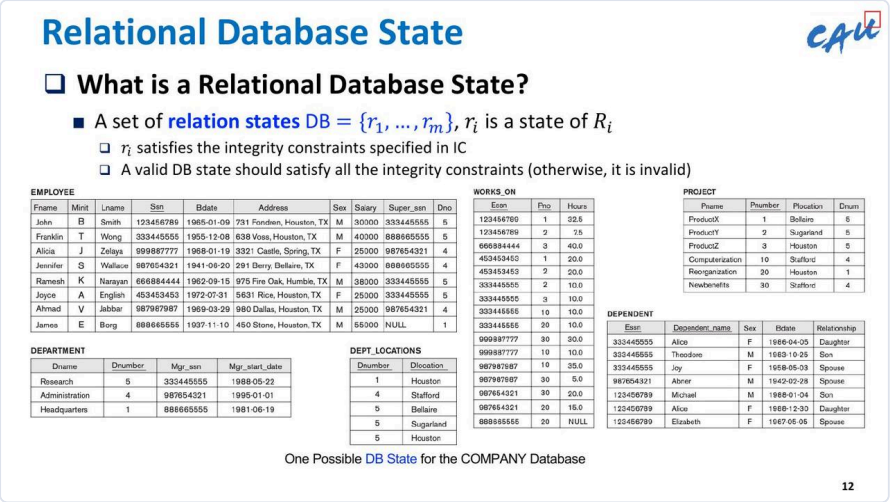
---

§ Relational database schema and state

- **Relational database schema S** = a set of relation schemas {R_1, ..., R_m} + a **set of integrity constraints IC**
  - e.g., COMPANY = {EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT}
  - In the schema diagram, **underline = primary key**, **arrow = referential integrity** (FK → referenced PK)
- **Relational database state DB** = a set of relation states {r_1, ..., r_m}, where r_i is a state of R_i
  - If it satisfies all integrity constraints it is a **valid state**; if it violates even one, it is an **invalid state**



03 p.11 · COMPANY relational database schema (underline = PK, arrow = referential integrity)



03 p.12 · COMPANY database state (the data used throughout the practice problems)

Keys and Integrity Constraints

Slides: 3-1 p.13–21

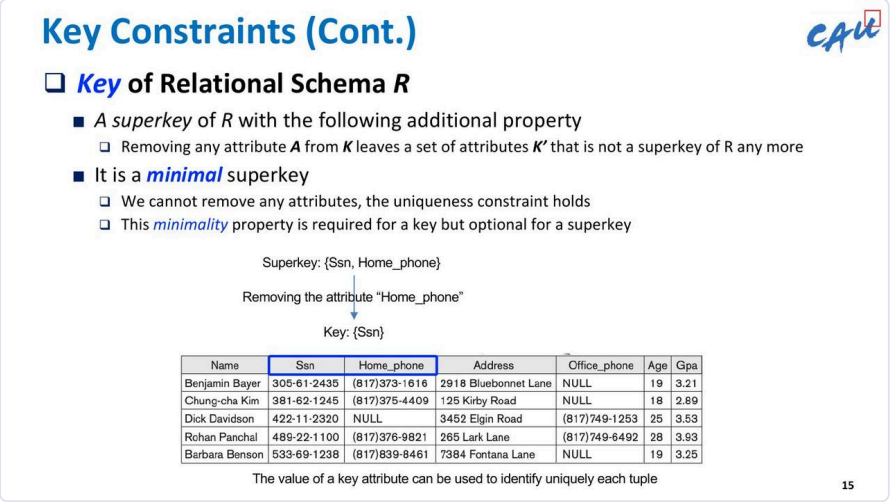
§ Superkey and key

A DBMS must provide ways to define and enforce constraints. The three core constraints = **key constraints, entity integrity, referential integrity**.

Key constraint: "no two distinct tuples can have the same value for the SK attributes" → in every state r, $t_1[SK] \neq t_2[SK]$.

- **Superkey (SK)**: a **subset of attributes that uniquely identifies each tuple**. Every relation has at least one default superkey (the set of all its attributes)
- **Key (K)**: a superkey such that removing any attribute A leaves a set that is no longer a superkey = a **minimal superkey**
  - Minimality is required for a key, optional for a superkey
  - e.g., {Ssn, Home_phone} is a superkey. Even after removing Home_phone, {Ssn} is unique → {Ssn, Home_phone} is not a key; {Ssn} is a key

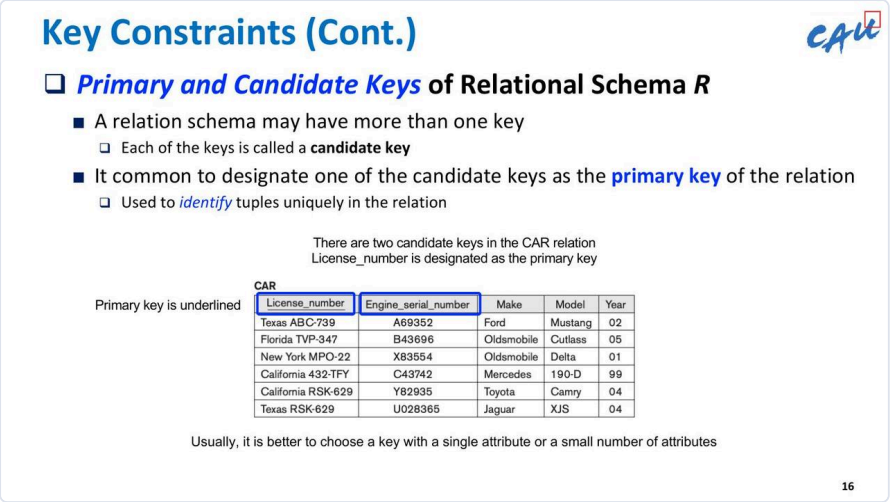
Trap: key ⊂ superkey. "Every key is a superkey, but not every superkey is a key."



03 p.15 · Removing Home_phone from superkey {Ssn, Home_phone} leaves key {Ssn}

§ Candidate key and primary key

- If a relation schema has more than one key, each is called a **candidate key**
- One of the candidate keys is designated as the **primary key** and used to identify tuples. **Underlined** in the schema
- e.g., CAR's candidate keys = License_number, Engine_serial_number → License_number is chosen as the PK
- It is usually better to choose a PK with **a single attribute or a small number of attributes**



03 p.16 · Of CAR's two candidate keys, License_number is chosen as the PK

§ Entity integrity constraint

"No primary key value can be NULL" → for every tuple t, t[PK] ≠ NULL.

- Reason: if two or more tuples had NULL primary keys, we **could not distinguish them** when referencing them from other relations
- e.g., an attribute that has NULLs, such as Office_phone, cannot be a key
- For a composite PK, **no part of the PK** may be NULL

§ Referential integrity constraint

"The FK value of tuple t_1 must either equal the PK value of some tuple t_2, or be NULL"

- **Foreign key (FK)**: a set of attributes in relation R_1 that **references** R_2. If $t_1[FK] = t_2[PK]$, t_1 references t_2
- The FK of R_1 has the **same domain** as the PK of R_2
- R_1 = the **referencing relation**, R_2 = the **referenced relation**
- e.g., EMPLOYEE.Dno → DEPARTMENT.Dnumber. 'John Smith' has Dno=5 → references the 'Research' department tuple = Smith works for Research
- Why it is needed: to maintain **consistency** among tuples of two relations. If Dno=3 but department 3 does not exist, the value cannot be interpreted
- An FK can **reference its own relation** (EMPLOYEE.Super_ssn → EMPLOYEE.Ssn)

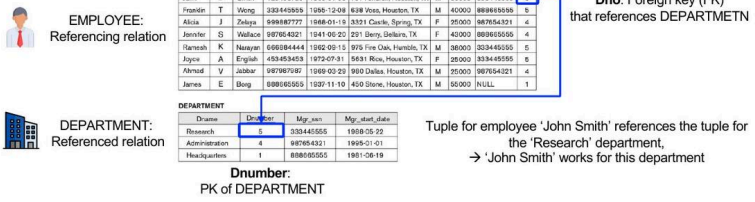
Slide exercise (find the FKs):

- SALESPERSON(Ssn, Name, Start_year, Dept_no), TRIP(Ssn, From_city, To_city, Departure_date, Return_date, Trip_id), EXPENSE(Trip_id, Account#, Amount)
  → TRIP.Ssn → SALESPERSON, EXPENSE.Trip_id → TRIP
- STUDENT(Ssn, ...), COURSE(Course#, ...), ENROLL(Ssn, Course#, Quarter, Grade), BOOK_ADOPTION(Course#, Quarter, Book_isbn), TEXT(Book_isbn, ...)
  → ENROLL.Ssn → STUDENT, ENROLL.Course# → COURSE, BOOK_ADOPTION.Course# → COURSE, BOOK_ADOPTION.Book_isbn → TEXT

Referential Integrity Constraint

❑ What is a *Referential Integrity Constraint*?

- ***"A value of FK in a tuple t_1 either has a value of PK for some tuple t_2 or is NULL"***
  - *Foreign Key (FK):* a set of attributes in relation schema R_1 **references** R_2
  - i.e., $t_1(FK) = t_2(PK)$ (the tuple t_1 **references** the tuple t_2)
- The FK of R_1 has the same domain(s) as the PK of R_2



EMPLOYEE:
Referencing relation

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Dno: Foreign key (FK)
that references DEPARTMETN

DEPARTMENT:
Referenced relation

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Dnumber:
PK of DEPARTMENT

Tuple for employee 'John Smith' references the tuple for the 'Research' department,
→ 'John Smith' works for this department

18

03 p.18 · EMPLOYEE.Dno (FK) → DEPARTMENT.Dnumber (PK)

Update Operations and Constraint Violations

Slides: 3-1 p.23–30

§ Insert

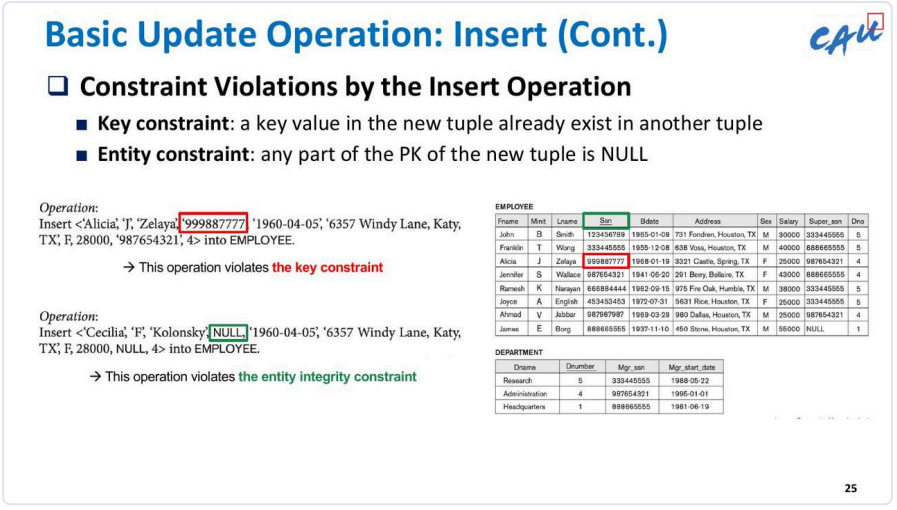
Kinds of operations: **retrievals** and **updates**. Basic update operations = Insert, Delete, Update (Modify). Each one may violate constraints when applied.

Insert: adds one tuple to a relation. Given the relation name + a list of values (in attribute order).

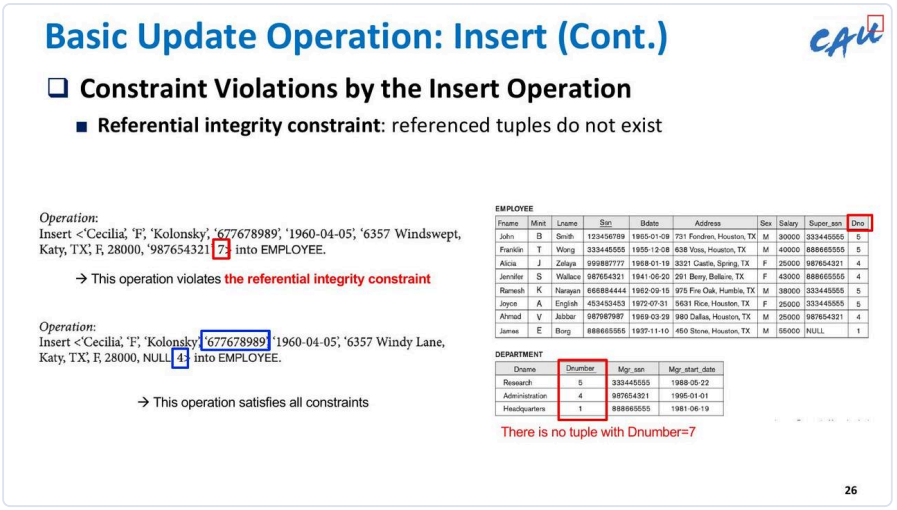
Insert can violate all three constraints.

Violation	Cause	Slide example (COMPANY state)
Key constraint	The new tuple's key value already exists in another tuple	<'Alicia', ..., **'999887777'**, ..., 4> → Ssn 999887777 already belongs to Zelaya
Entity integrity	The new tuple's PK (or part of it) is NULL	<'Cecilia', ..., **NULL**, ..., 4>
Referential integrity	The tuple referenced by the FK does not exist	<'Cecilia', ..., '677678989', ..., **7**> → no department with Dnumber=7

<'Cecilia', 'F', 'Kolonsky', '677678989', ..., NULL, 4> **satisfies all constraints** (Super_ssn being NULL is allowed because it is an FK).



03 p.25 · Insert violating the key constraint and entity integrity



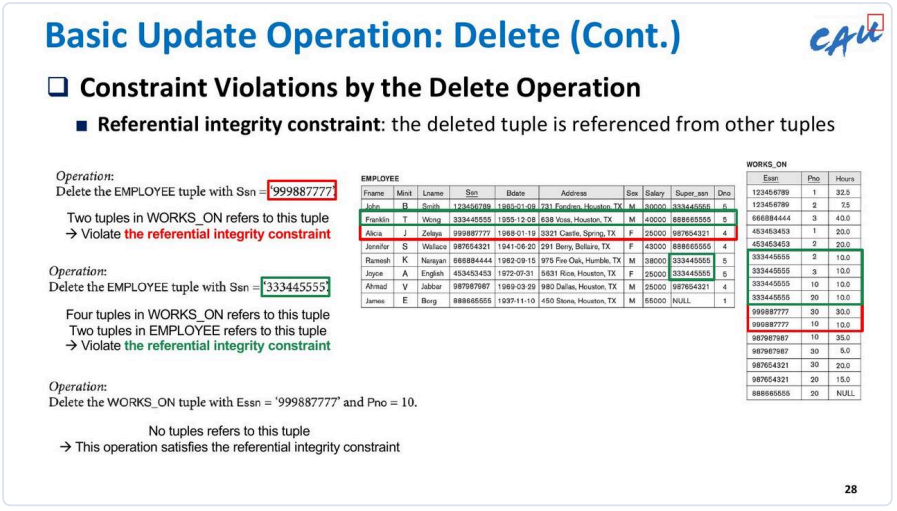
03 p.26 · Insert violating referential integrity (Dno=7) and a case satisfying all constraints

§ Delete

Delete: specify the relation name + the PK value of the tuple to delete.

Delete can violate only referential integrity (when the deleted tuple is referenced by other tuples).

- Delete Ssn='999887777' from EMPLOYEE → referenced by 2 WORKS_ON tuples → **violation**
- Delete Ssn='333445555' from EMPLOYEE → referenced by 4 WORKS_ON tuples + 2 EMPLOYEE tuples (Super_ssn) → **violation** (in fact DEPARTMENT.Mgr_ssn and DEPENDENT.Essn also reference it)
- Delete Essn='999887777' AND Pno=10 from WORKS_ON → nothing references it → OK



03 p.28 · Delete violating referential integrity

§ Update (Modify)

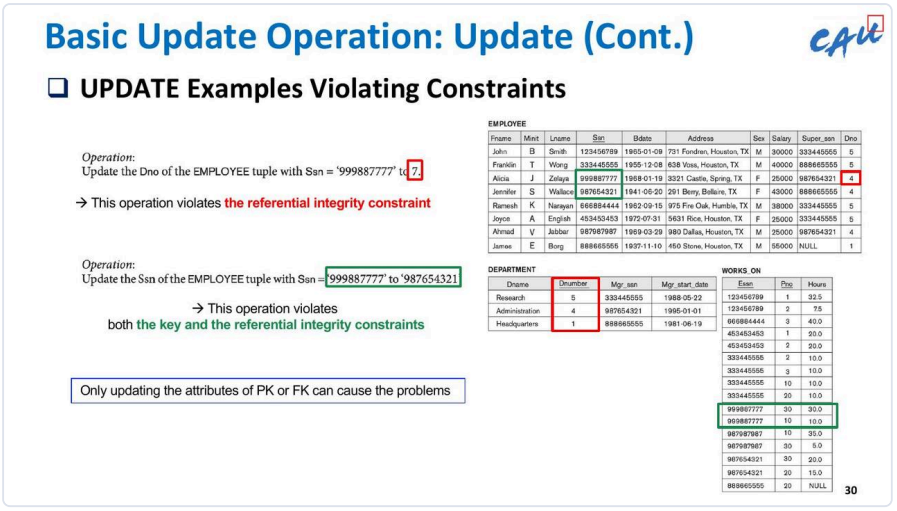
Update: specify the attribute(s) to change and the new value(s).

Update can violate key constraints and referential integrity. It is a problem only when a PK or FK attribute is modified.

- Change Dno of the employee with Ssn='999887777' to **7** → no department 7 → **referential integrity** violation

- Change Ssn='999887777' to **'987654321'** → already Wallace's Ssn (key constraint) + tuples in WORKS_ON etc. that referenced 999887777 become orphans (referential integrity) → **both** violated
- Changing a PK to NULL also violates entity integrity (the slide summary lists only key and referential)

> Exam point: exactly as in the summary table — Insert = key, entity and referential **all possible**; Delete = **referential only**; Update = **key and referential** (only when changing a PK/FK).



03 p.30 · Update violating referential integrity, and violating key + referential integrity at once

ER-to-Relational Mapping ①: Entities, Multivalued Attributes, Weak Entities

Slides: 3-1 p.32–36

§ Mapping regular entity types

Mapping order: entity types (regular, weak) → multivalued attributes → relationship types (1:1, 1:N, M:N, n-ary).

Regular entity type E

1. Create a relation R that includes **all simple attributes** of E. For a composite attribute include **only its simple component attributes** (Name → Fname, Minit, Lname)
2. Choose one of E's key attributes as the **PK** of R (other keys become UNIQUE candidate keys)
3. Multivalued attributes are not included here → next step

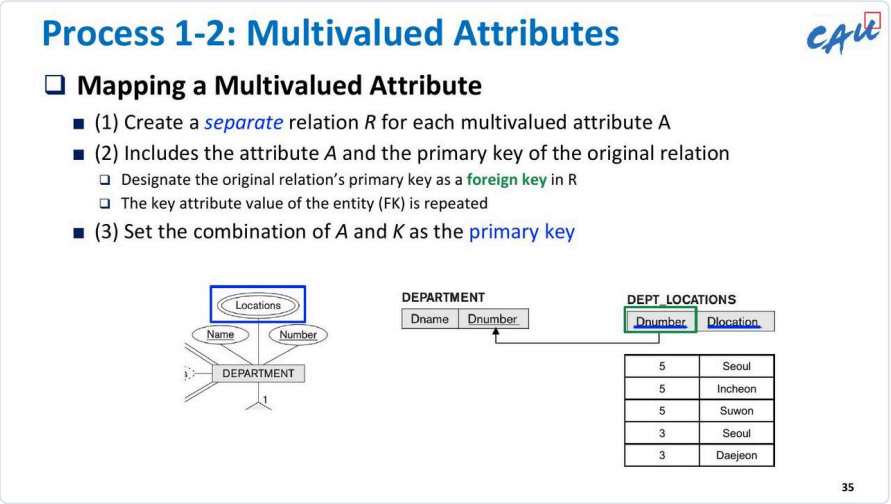
e.g., DEPARTMENT(Dname, **Dnumber**) — Locations is left out for now.

§ Mapping multivalued attributes

Multivalued attribute A (not allowed in a relation)

1. Create a **separate relation R** for each A
2. Put A and the PK K of the original relation into R. K is an **FK** in R (referencing the original relation). Key values repeat
3. The **combination (A, K) is the PK**

e.g., DEPT_LOCATIONS(**Dnumber**, **Dlocation**), Dnumber → DEPARTMENT. (5, Seoul), (5, Incheon), (5, Suwon), (3, Seoul), (3, Daejeon)



03 p.35 · Multivalued attribute Locations → DEPT_LOCATIONS(Dnumber, Dlocation)

§ Mapping weak entity types

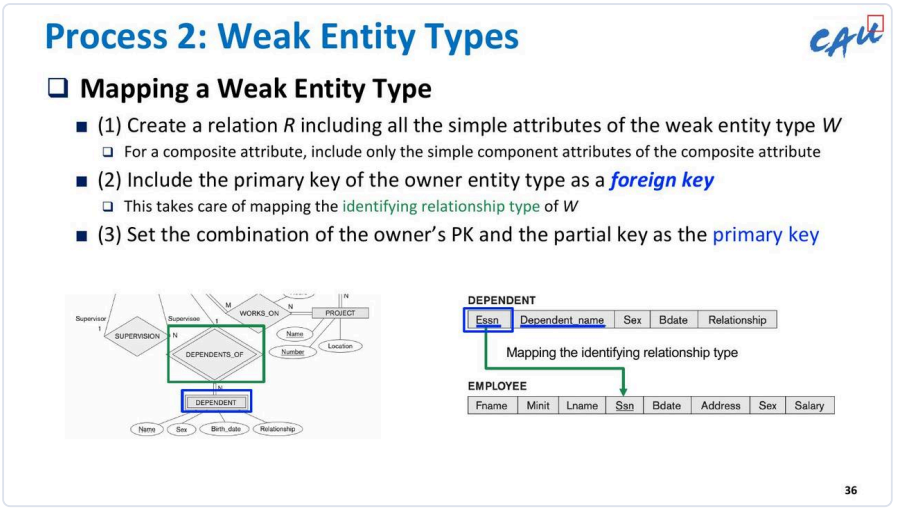
Weak entity type W

1. Create a relation R that includes all simple attributes of W (only component attributes of composites)

2. Include the **PK of the owner entity type as an FK** → this is the **mapping of the identifying relationship type**
3. The **combination of the owner's PK + the partial key is the PK**

e.g., DEPENDENT(**Essn**, **Dependent_name**, Sex, Bdate, Relationship), Essn → EMPLOYEE.Ssn

Trap: the identifying relationship (DEPENDENTS_OF) does not get its own relation. Putting the owner's PK in as an FK is itself the mapping of the identifying relationship.



03 p.36 · Weak entity DEPENDENT → PK (Essn, Dependent_name)

ER-to-Relational Mapping ②: Relationship Types

Slides: 3-1 p.37–49

§ Mapping 1:1 relationships

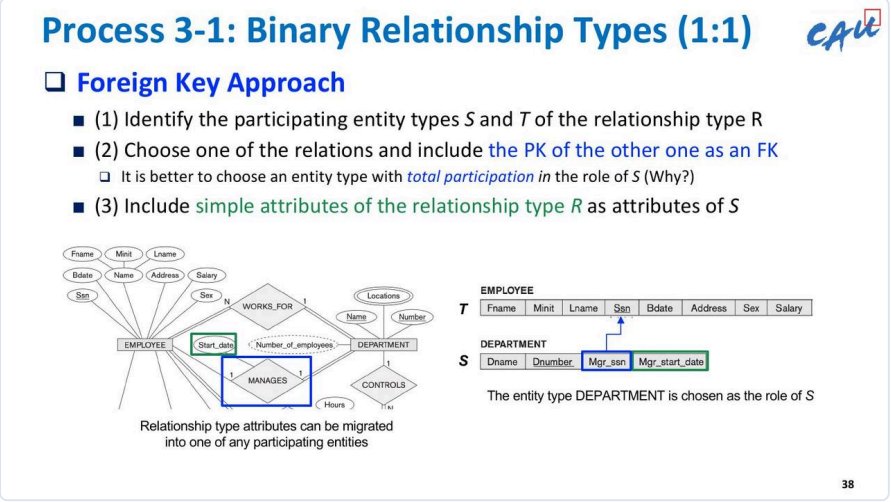
Three approaches: **foreign key approach** (most common and useful), **merged relation approach** (combine the two entities into one relation), **cross-reference (relationship relation) approach**.

Foreign key approach

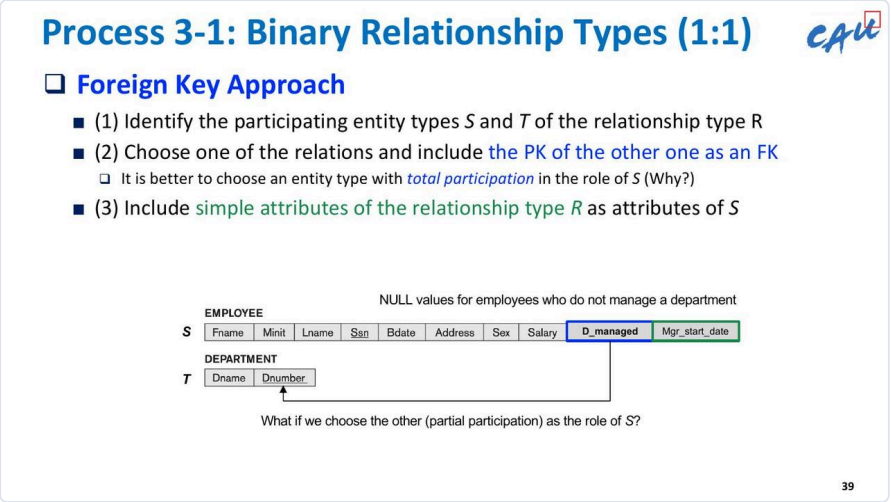
1. Identify the participating entity types S and T
2. Choose one side (S) and include **the PK of the other side (T) as an FK**. It is better to choose **the side with total participation as S**
3. Also include the simple attributes of the relationship in S

e.g., MANAGES: choose DEPARTMENT (total participation) as S → DEPARTMENT(Dname, **Dnumber**, Mgr_ssn, Mgr_start_date), Mgr_ssn → EMPLOYEE.

Why the total-participation side? If we instead choose EMPLOYEE (partial participation) as S, EMPLOYEE gets D_managed and Mgr_start_date, and for most employees, who do not manage a department, these values would be **NULL**. Putting them on the total-participation side produces no NULLs.



03 p.38 · 1:1 MANAGES: Mgr_ssn, Mgr_start_date in DEPARTMENT, the total-participation side



03 p.39 · Choosing EMPLOYEE, the partial-participation side, produces many NULLs

§ Mapping 1:N relationships

Two approaches: **foreign key approach** (preferred because it reduces the number of relations), cross-reference approach.

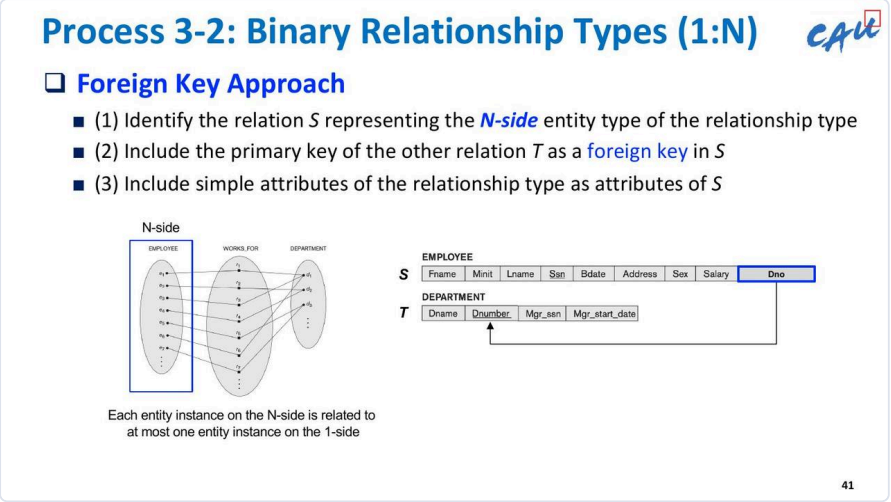
Foreign key approach

1. Identify the relation S that represents the entity type on the **N-side**
2. Include in S the **PK of the other (1-side) relation T as an FK**
3. Also include the simple attributes of the relationship in S

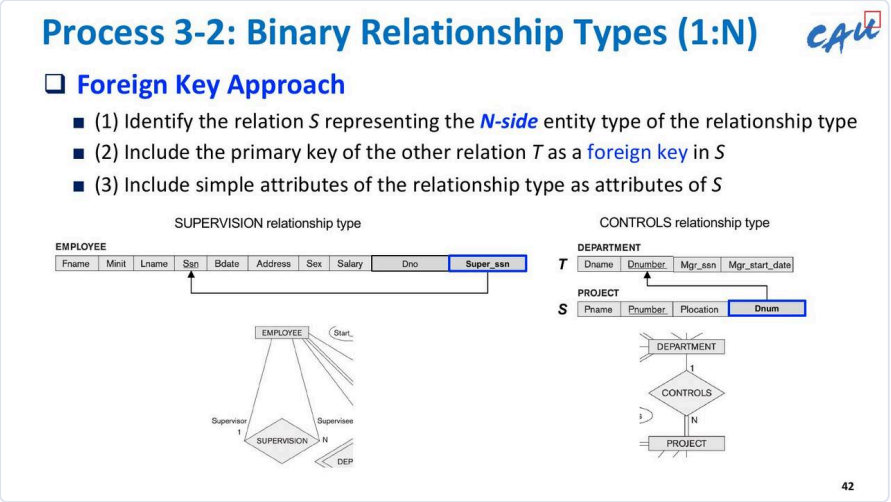
Reason: one N-side entity instance is related to **at most one** 1-side entity → can be represented by a single FK value.

- WORKS_FOR → **Dno** in EMPLOYEE (→ DEPARTMENT)
- SUPERVISION → **Super_ssn** in EMPLOYEE (→ EMPLOYEE, self-reference)
- CONTROLS → **Dnum** in PROJECT (→ DEPARTMENT)

Trap: putting the FK on the 1-side makes it multivalued, violating atomicity. The FK **always goes on the N-side**.



03 p.41 · 1:N WORKS_FOR: Dno in EMPLOYEE, the N-side



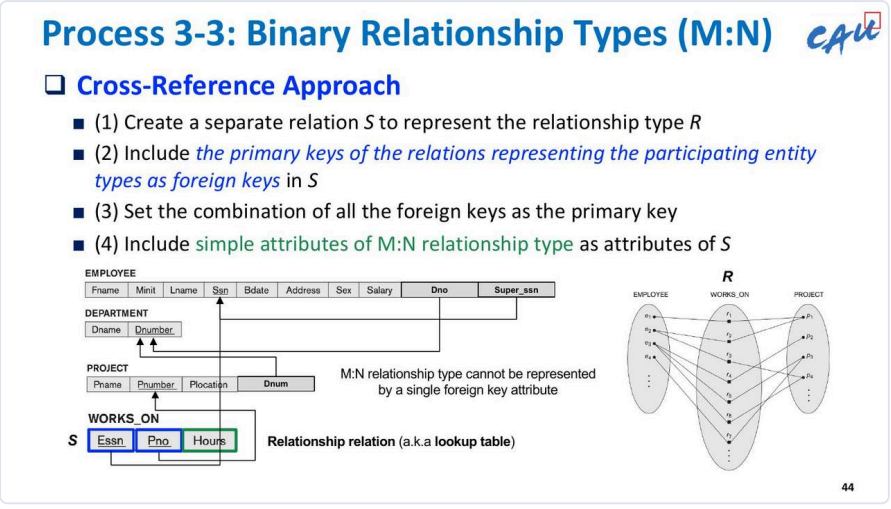
03 p.42 · SUPERVISION → Super_ssn, CONTROLS → Dnum

§ Mapping M:N relationships

Only one approach: the **cross-reference (relationship relation) approach**. An M:N relationship cannot be represented by a single FK.

1. Create a **separate relation S** to represent relationship type R
2. Include the **PKs of the participating entity relations as FKs** in S
3. The **combination of the FKs is the PK**
4. Also include the simple attributes of the M:N relationship in S

e.g., WORKS_ON(**Essn**, **Pno**, Hours), Essn → EMPLOYEE, Pno → PROJECT. Relationship relation = lookup table.



03 p.44 · M:N WORKS_ON → relationship relation WORKS_ON(Essn, Pno, Hours)

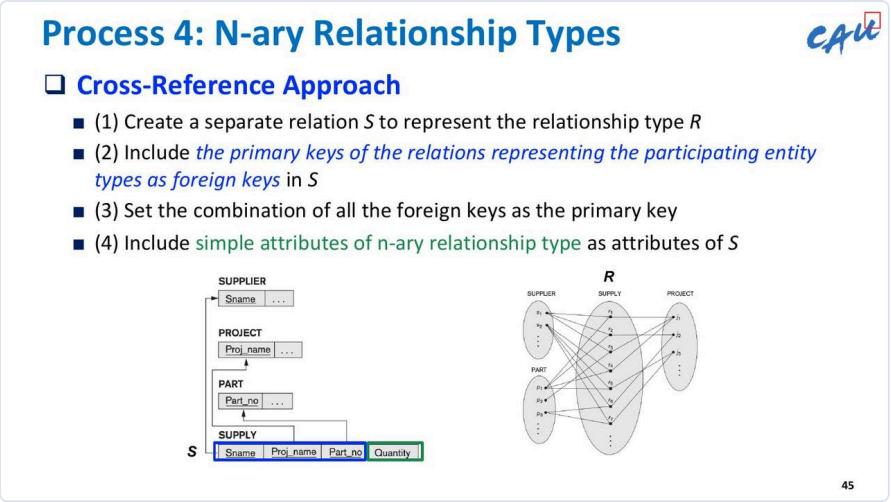
§ Mapping n-ary relationships

Cross-reference approach (same procedure as M:N)

1. Create a separate relation S
2. Include the PKs of the n participating entity relations as FKs

3. The combination of all FKs is the PK (usually; if some participation has max = 1, that FK can be left out of the PK)
4. Include the simple attributes of the relationship

e.g., SUPPLY(**Sname**, **Proj_name**, **Part_no**, Quantity) — references SUPPLIER, PROJECT, PART.



03 p.45 · Ternary SUPPLY → SUPPLY(Sname, Proj_name, Part_no, Quantity)

§ ER ↔ relational model correspondence

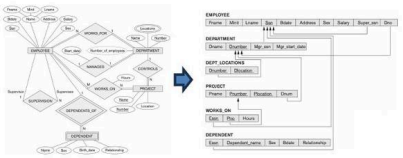
ER model	Relational model
Entity type	Entity relation
1:1 or 1:N relationship type	Foreign key (or relationship relation)
M:N relationship type	Relationship relation + **two** FKs
n-ary relationship type	Relationship relation + **n** FKs
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation + foreign key
Value set	Domain
Key attribute	Primary key (or secondary key)

Slide Exercise 3 (UNIVERSITY: COLLEGE, INSTRUCTOR, DEPT, STUDENT, COURSE, SECTION), solution order:

1. Regular entities → COLLEGE(CName, COffice, CPhone), INSTRUCTOR(Id, Rank, IName, IOffice, IPhone), DEPT(DName, DCode, DOffice, DPhone), COURSE(CCode, Credits, CoName, Level, CDesc), SECTION(SecId, SecNo, Sem, Year, Bldg, RoomNo, DaysTime), STUDENT(Sid, DOB, FName, MName, LName, Addr, Phone, Major)
2. 1:1 → DEAN: DeanId in COLLEGE (COLLEGE has (1,1) total participation); CHAIR: ChairId, CStartDate in DEPT
3. 1:N → INSTRUCTOR.Employed (EMPLOYS), DEPT.Admined (ADMINS), COURSE.Offered (OFFERS), SECTION.Taught (TEACHES), SECTION.Belongs (SECS), STUDENT.Dept (HAS)
4. M:N → TAKES(**Sid**, **SecId**, Grade)

Takeaways: ER-to-Relational Mapping

❑ **Correspondence between ER and Relational Models**



ER MODEL	RELATIONAL MODEL
Entity type	*Entity* relation
1:1 or 1:N relationship type	Foreign key (or *relationship* relation)
M:N relationship type	*Relationship* relation and *two* foreign keys
n-ary relationship type	*Relationship* relation and *n* foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

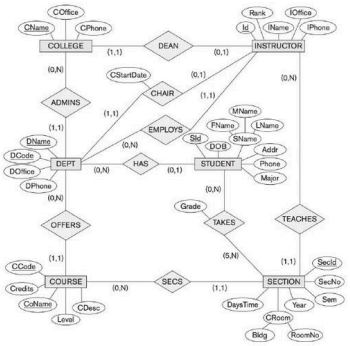
49

03 p.49 · Correspondence table between the ER model and the relational model

Exercises: ER-to-Relational Mapping

❑ **Exercise 3:**

- Map the ER schema for UNIVERSITY database into a relational database schema

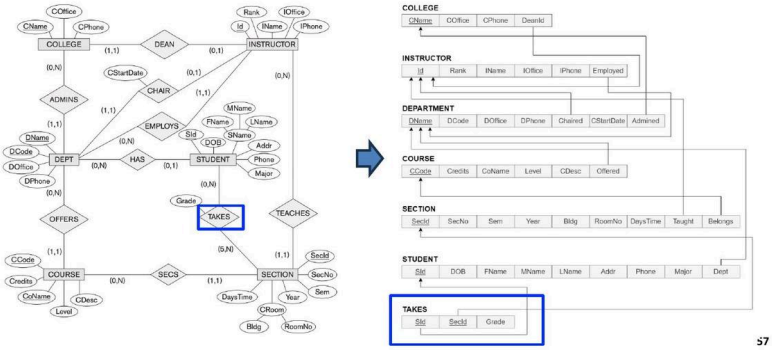


46

03 p.46 · Exercise 3: UNIVERSITY ER schema

Exercises: ER-to-Relational Mapping (Cont.)

❑ **Step 4: Mapping of Binary Relationship Types (M:N)**



57

db-exercises p.57 · Exercise 3 mapping result (including M:N TAKES)

U4 · 4-1_SQL.pdf + 3-2_setup-for-sql.pdf + SQL-week6-lab.pdf

Basic SQL (DDL, DML, SELECT)

Constraint syntax in CREATE TABLE, referential triggered actions (ON DELETE/UPDATE), what INSERT, DELETE and UPDATE can violate, and SELECT-FROM-WHERE. You only get better at SQL by writing it. The practice answers include the actual results of running each query on the textbook COMPANY state (3-1 p.12).

L4-1 · BASIC SQL (DDL, DML, SELECT)

Introduction to SQL, CREATE SCHEMA/TABLE, Constraints

Slides: 4-1 p.3–10

§ SQL and NL2SQL

- **SQL (Structured Query Language)**: a special-purpose **declarative** language. Designed for IBM's first DBMS; originally called **SEQUEL** (Structured English Query Language). Pronounced 'S-Q-L' or 'SEQUEL'
- **DDL (Data Definition Language)**: commands on databases and **tables**. CREATE / ALTER / DROP (schema, table)
- **DML (Data Manipulation Language)**: commands on **tuples** in tables. Retrieval: SELECT; modification: INSERT / DELETE / UPDATE
- **NL2SQL**: technology that translates natural-language queries into SQL. Challenges = ambiguity of natural language, schema understanding and linking (schema linking), generalization to unseen databases, generating complex queries. Even LLM-based methods are still far from perfect → **understanding SQL properly matters**

> **Lecture note** Week 3-2 (TA)
> Lab environment: VS Code + Python (about 3.11), pip install pandas sqlglot. pandas saves SQL results as CSV; sqlglot checks SQL syntax. Do not modify the code in test.py except replacing 'change me' with your student ID. Read NL_queries.txt, write SQL in mysql.py and run test.py; a folder named with your student ID is created — zip it and submit it to e-class. The zip file name must be your student ID.

§ CREATE SCHEMA / CREATE TABLE

SQL schema: the elements that describe a schema — tables, types, constraints, views, domains, etc. Identified by a schema name.

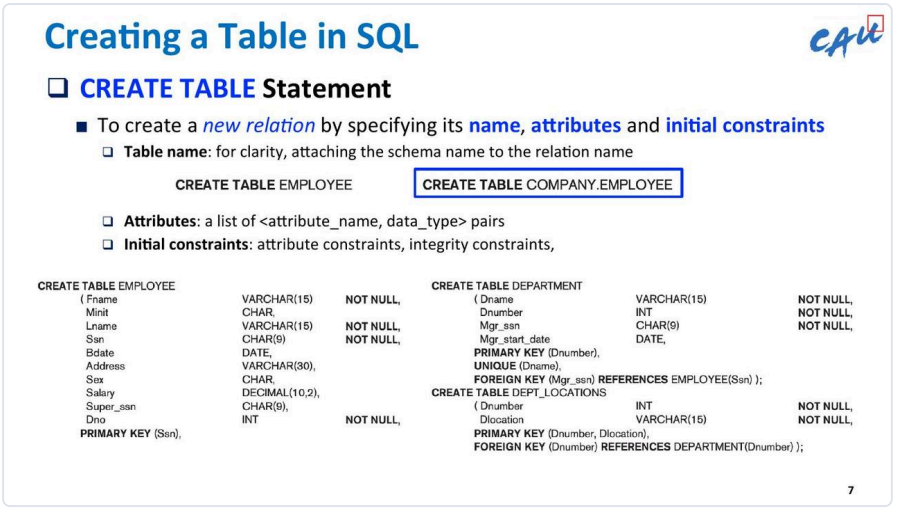
```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

- **Authorization identifier**: the user who owns the schema

CREATE TABLE: creates a new relation by giving its name, attributes and initial constraints.

- The table name can be prefixed with the schema name: `CREATE TABLE COMPANY.EMPLOYEE`
- Attributes = a list of <attribute name, data type> pairs
- Initial constraints = attribute constraints, integrity constraints

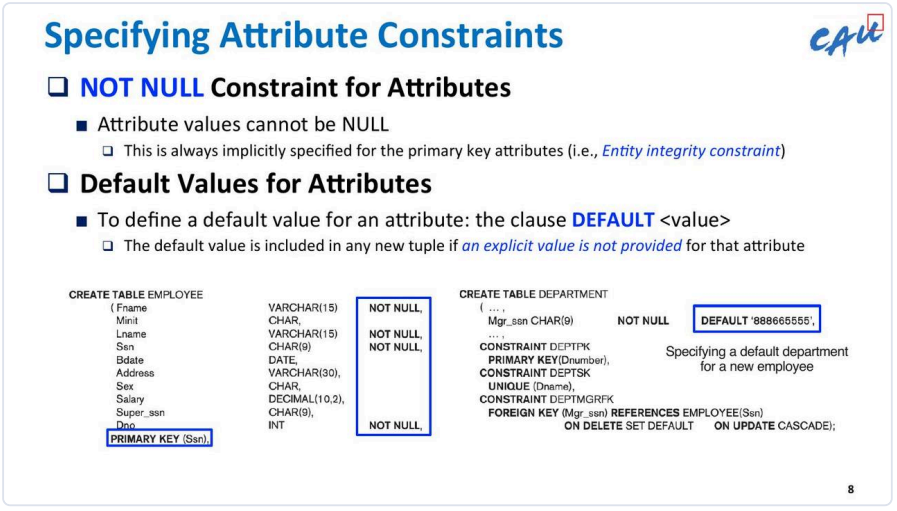
```
CREATE TABLE EMPLOYEE
( Fname      VARCHAR(15)    NOT NULL,
  Minit      CHAR,
  Lname      VARCHAR(15)    NOT NULL,
  Ssn        CHAR(9)        NOT NULL,
  Bdate      DATE,
  Address    VARCHAR(30),
  Sex        CHAR,
  Salary     DECIMAL(10,2),
  Super_ssn  CHAR(9),
  Dno        INT            NOT NULL,
  PRIMARY KEY (Ssn) );
```



04 p.7 · CREATE TABLE EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS

§ Attribute constraints: NOT NULL, DEFAULT

- **NOT NULL**: the attribute's value cannot be NULL. **Always implicitly** applied to PK attributes (= entity integrity)
- **DEFAULT <value>**: if a new tuple does not give a value for the attribute, the default value is used
  - e.g., a new employee's default department: `Dno INT NOT NULL DEFAULT 1`; in DEPARTMENT, `Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555'`



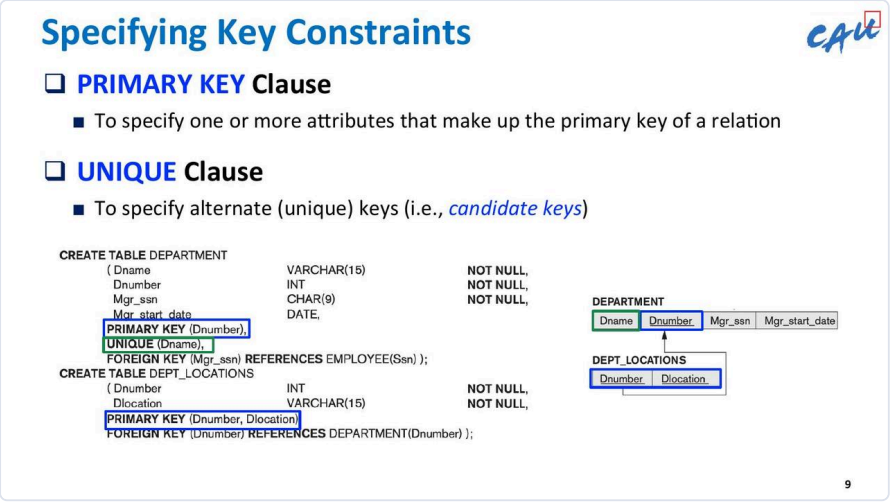
04 p.8 · NOT NULL and DEFAULT

§ Key constraints: PRIMARY KEY, UNIQUE, FOREIGN KEY

```
CREATE TABLE DEPARTMENT
( Dname           VARCHAR(15)  NOT NULL,
  Dnumber         INT          NOT NULL,
  Mgr_ssn         CHAR(9)      NOT NULL,
  Mgr_start_date  DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber    INT          NOT NULL,
  Dlocation  VARCHAR(15)  NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

- **PRIMARY KEY**: the attribute(s) that make up the PK. If composite, list several inside the parentheses
- **UNIQUE**: an alternate key = a **candidate key**
- **FOREIGN KEY ... REFERENCES table(attribute)**: an FK that references the PK of another relation
- Naming a constraint: `CONSTRAINT DEPTPK PRIMARY KEY (Dnumber)` → the name is needed to drop it later with ALTER

> Exam point: referential integrity can be violated when a referenced PK is updated or deleted; **SQL's default action is to reject** the operation. The alternative = referential triggered actions (L4-2).



04 p.9 · PRIMARY KEY, UNIQUE, FOREIGN KEY

Referential Triggered Actions, DROP, ALTER

Slides: 4-1 p.11–20

§ Referential triggered actions

An action attached to an FK that answers **"what happens when a tuple referenced by other tuples is deleted/updated?"**

- **Triggers**: `ON DELETE`, `ON UPDATE`
- **Actions**: `SET NULL`, `CASCADE`, `SET DEFAULT` (if none is given, the default = reject; called REJECT in the slide exercise)

Action	When the referenced tuple is deleted	When it is updated
SET NULL	Set the referencing FK to NULL	Set the FK to NULL
SET DEFAULT	Set the FK to its default value	Set the FK to its default value
CASCADE	**Delete the referencing tuples too**	Change the referencing FK **to the new value**

```
CREATE TABLE EMPLOYEE
( ...,
  Dno INT NOT NULL DEFAULT 1,
  CONSTRAINT EMPPK PRIMARY KEY (Ssn),
  CONSTRAINT EMPSUPERFK FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
      ON DELETE SET NULL      ON UPDATE CASCADE,
  CONSTRAINT EMPDEPTFK  FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
      ON DELETE SET DEFAULT   ON UPDATE CASCADE );
```

Slide discussion (p.13): what happens if EMPSUPERFK's ON DELETE is changed to **CASCADE** and `DELETE FROM EMPLOYEE WHERE Lname = 'Borg';` is executed?

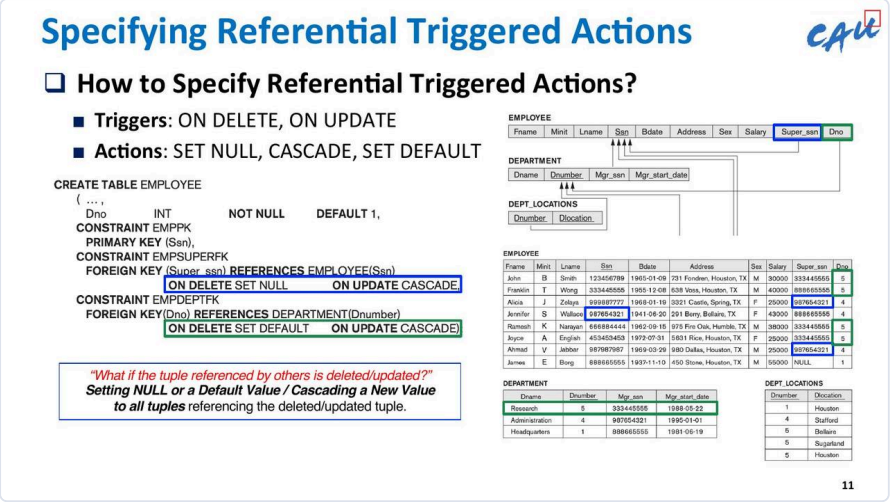
- Wong and Wallace, whose supervisor is Borg (888665555), are deleted too → then Smith, Narayan, English, Zelaya and Jabbar, whose supervisors are Wong and Wallace, are deleted in a chain → **all 8 EMPLOYEE tuples disappear**. The key point is that CASCADE propagates recursively
- (This considers only EMPSUPERFK. In reality, if other FKs such as WORKS_ON.Essn and DEPENDENT.Essn use the default action (reject), those tuples reference employees that would be deleted, so the whole DELETE is rejected)

Slide Exercise 1 (LIBRARY) sample answer:

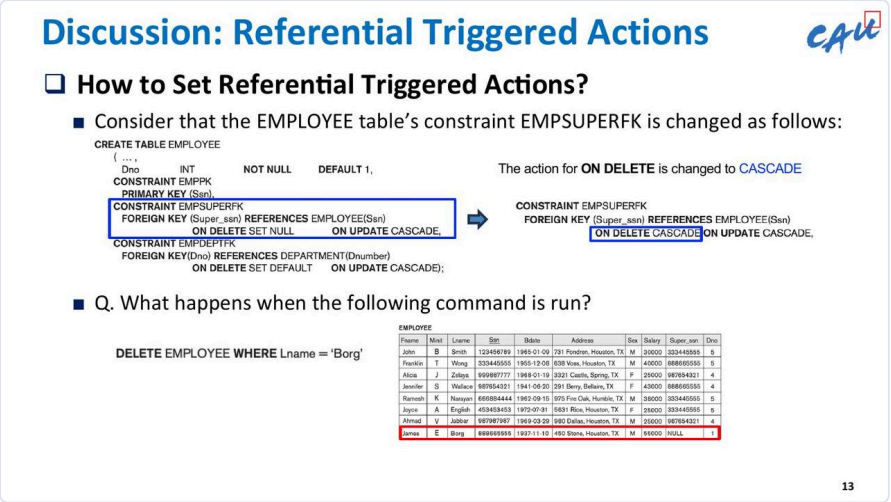
FK	ON DELETE	ON UPDATE	Reason
BOOK_AUTHORS.Book_id → BOOK	CASCADE	CASCADE	If a book is deleted, its author info is no longer needed
BOOK.Publisher_name → PUBLISHER	REJECT	CASCADE	Do not allow deleting a publisher that still has books
BOOK_COPIES.Book_id → BOOK	CASCADE	CASCADE	If a book is removed from the catalog, its copies are no longer managed

FK	ON DELETE	ON UPDATE	Reason
BOOK_COPIES.Branch_id → LIBRARY_BRANCH	CASCADE	CASCADE	Deleting a branch = the branch closes
BOOK_LOANS.Book_id → BOOK	REJECT (or CASCADE)	CASCADE	Even if removed from the catalog, loaned books must be returned
BOOK_LOANS.Branch_id → LIBRARY_BRANCH	CASCADE (or REJECT)	CASCADE	When a branch closes, its loans are forgiven
BOOK_LOANS.Card_no → BORROWER	REJECT	CASCADE	A borrower can leave only after returning all books

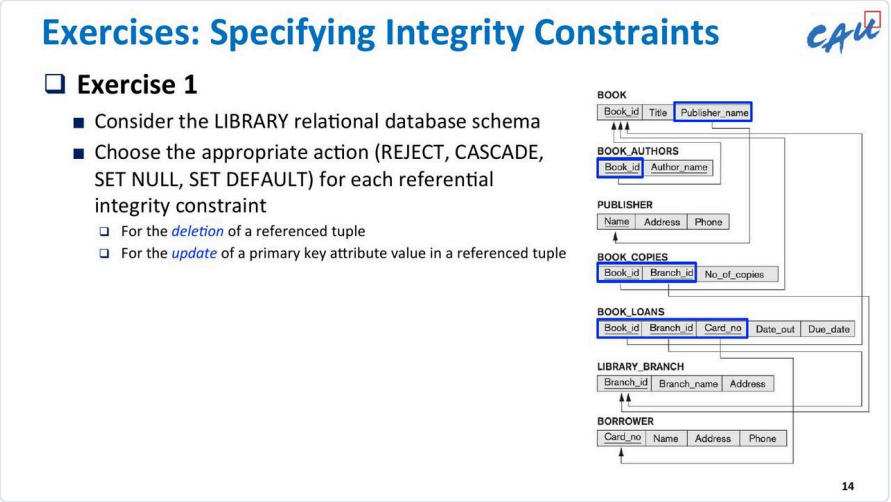
> Exam point: there is no single right answer; an answer is accepted **if the reason is sound**. ON UPDATE is almost always CASCADE (when a key changes, references must follow it to stay traceable).



04 p.11 · ON DELETE SET NULL / SET DEFAULT, ON UPDATE CASCADE



04 p.13 · Discussion: change EMPSUPERFK to ON DELETE CASCADE and delete Borg



04 p.14 · Exercise 1: LIBRARY schema

§ DROP SCHEMA / DROP TABLE

Schema modification = changing the **schema**, not tuples (adding/dropping tables, attributes, constraints).

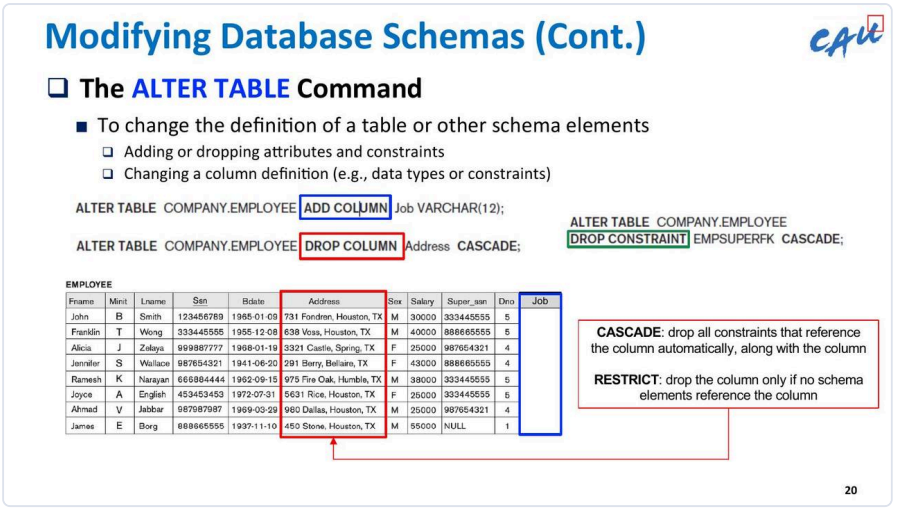
- **DROP SCHEMA**: drops an entire schema
  - `CASCADE`: also drops all elements of the schema (tables, domains, etc.)
  - `RESTRICT`: drops the schema **only if it has no elements** (the user must drop the elements one by one)
- **DROP TABLE**: drops a specific table. Removes all tuples + **the table definition from the catalog**
  - To delete only the tuples and keep the definition, use the **DELETE** command
  - `CASCADE`: automatically drops constraints and elements that reference the table too
  - `RESTRICT`: drops the table only if no constraint or element references it

§ ALTER TABLE

Changes a table definition: adding/dropping attributes and constraints, changing column definitions (type, constraints).

```
ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN Job VARCHAR(12);
ALTER TABLE COMPANY.EMPLOYEE DROP COLUMN Address CASCADE;
ALTER TABLE COMPANY.EMPLOYEE DROP CONSTRAINT EMPSUPERFK CASCADE;
```

- DROP COLUMN `CASCADE`: automatically drops constraints that reference the column too / `RESTRICT`: only if no element references the column
- A column added with ADD COLUMN is NULL (or the DEFAULT value) in existing tuples. So putting NOT NULL on a new column requires a DEFAULT



04 p.20 · ALTER TABLE ADD COLUMN / DROP COLUMN / DROP CONSTRAINT

INSERT, DELETE, UPDATE

Slides: 4-1 p.22–27

§ INSERT

Form 1: values for all attributes — list the values **in the same order** as the attributes were specified in CREATE TABLE

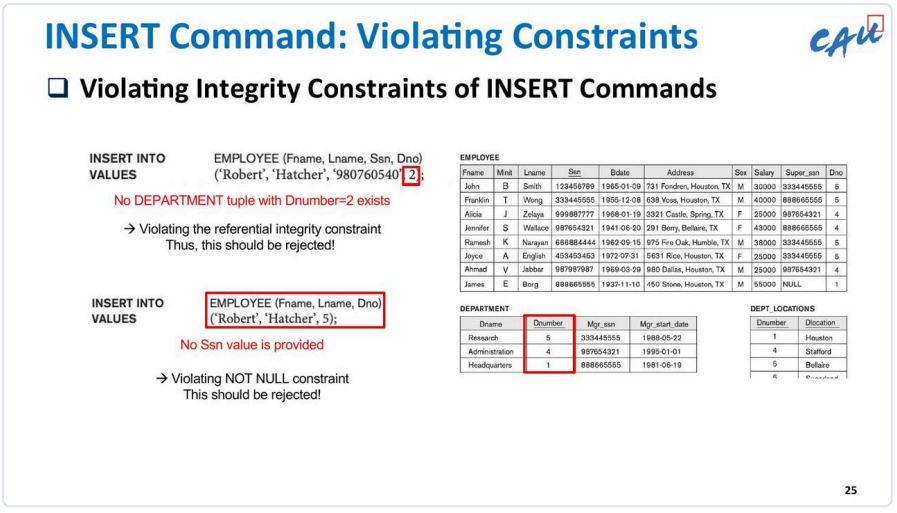
```
INSERT INTO EMPLOYEE
VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30',
        '98 Oak Forest, Katy, TX', 'M', 37000, '653298653', 4);
```

Form 2: explicit attribute names — attributes that are **NOT NULL and have no DEFAULT must** be included

```
INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard', 'Marini', 4, '653298653');
```

Constraint violation examples (rejected)

- `INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno) VALUES ('Robert', 'Hatcher', '980760540', 2);` → no department with Dnumber=2 → **referential integrity**
- `INSERT INTO EMPLOYEE (Fname, Lname, Dno) VALUES ('Robert', 'Hatcher', 5);` → no Ssn → **NOT NULL** (it is the PK, so entity integrity)



04 p.25 · INSERT violating referential integrity and NOT NULL

§ DELETE and UPDATE

DELETE: the WHERE clause selects the tuples to delete from **one relation**

- Explicitly deletes from only one table at a time. With a referential triggered action (ON DELETE CASCADE) it can propagate to other relations
- The number of tuples deleted varies (on the COMPANY state):

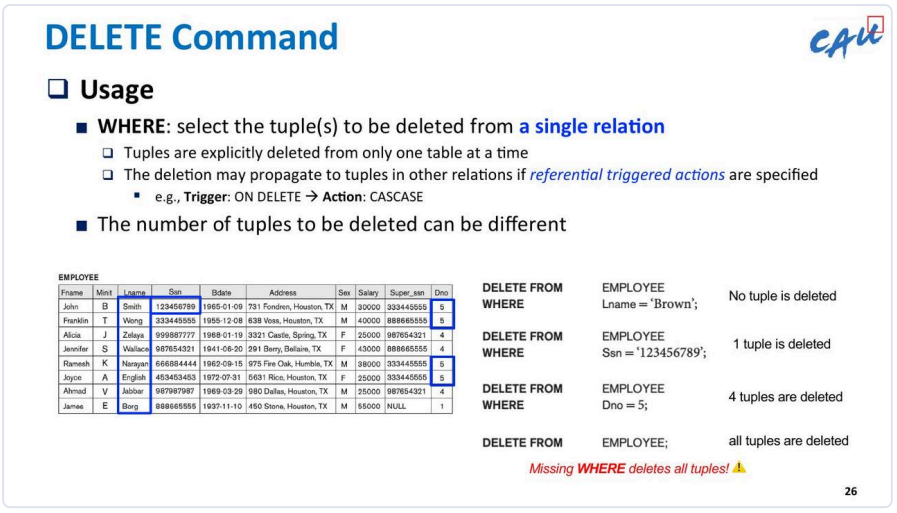
```
DELETE FROM EMPLOYEE WHERE Lname = 'Brown';        -- 0 tuples
DELETE FROM EMPLOYEE WHERE Ssn = '123456789';      -- 1 tuple
DELETE FROM EMPLOYEE WHERE Dno = 5;                -- 4 tuples
DELETE FROM EMPLOYEE;                              -- all of them!
```

Caution: **if WHERE is omitted, all tuples are deleted.** The table definition remains (unlike DROP TABLE).

UPDATE: the WHERE clause selects the tuples to modify in **one relation**, and **SET** specifies the attributes to change and their new values

```
UPDATE PROJECT  SET Plocation = 'Bellaire', Dnum = 5  WHERE Pnumber = 10;
UPDATE EMPLOYEE SET Salary = Salary * 1.1              WHERE Dno = 5;
```

- A single UPDATE can modify several tuples (the 4 employees with Dno=5 all get a 10% raise at once)
- With ON UPDATE CASCADE it propagates to other relations



04 p.26 · Number of tuples deleted by DELETE (0, 1, 4, all)

SELECT-FROM-WHERE, Aliases

Slides: 4-1 p.28–34

§ The SELECT-FROM-WHERE block

```
SELECT  <attribute list>      -- projection attributes
FROM    <table list>          -- relations needed by the query
WHERE   <condition>;          -- selection conditions + join conditions
```

Query 1: the birth date and address of 'John B. Smith'

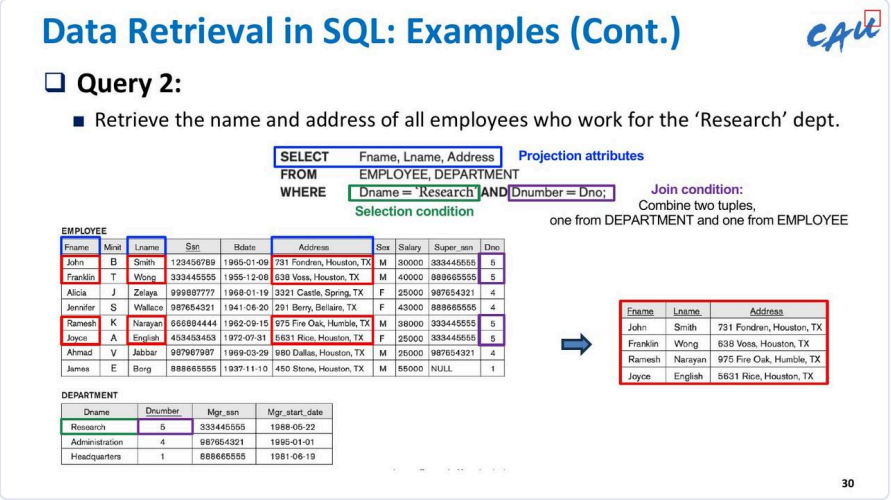
```
SELECT Bdate, Address
FROM   EMPLOYEE
WHERE  Fname = 'John' AND Minit = 'B' AND Lname = 'Smith';
```

→ 1965-01-09, 731 Fondren, Houston, TX

Query 2: the name and address of all employees who work for the 'Research' department

```
SELECT Fname, Lname, Address
FROM   EMPLOYEE, DEPARTMENT
WHERE  Dname = 'Research' AND Dnumber = Dno;
```

- `Dname = 'Research'` = **selection condition** (picks the tuples of interest from one table)
- `Dnumber = Dno` = **join condition** (combines DEPARTMENT tuples with EMPLOYEE tuples)
- Result: Smith, Wong, Narayan, English (4 employees)



04 p.30 · Query 2: selection condition Dname='Research' + join condition Dnumber=Dno

§ Ambiguous attribute names and aliases

Qualifying: if attributes in different relations have the same name, distinguish them as `RelationName.Attribute`.

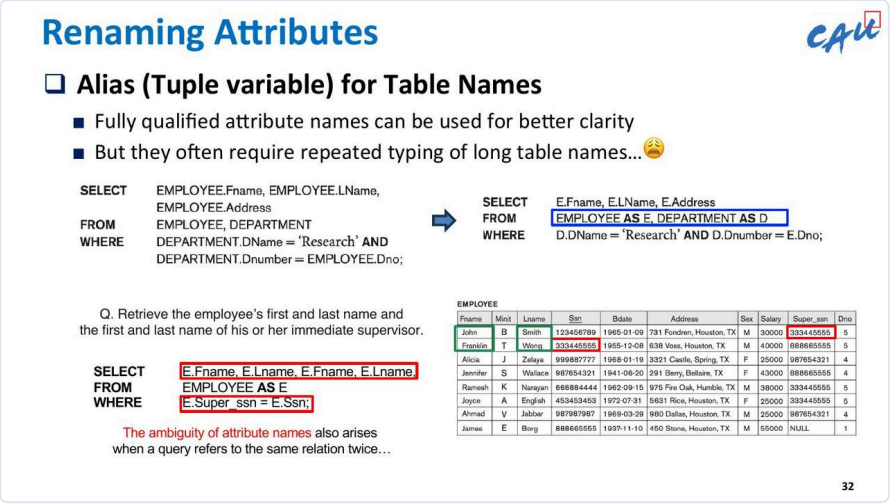
```
SELECT Fname, EMPLOYEE.Name, Address
FROM   EMPLOYEE, DEPARTMENT
WHERE  DEPARTMENT.Name = 'Research' AND DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

Alias (tuple variable): shortens repeated long names, and is required when referring to **the same relation twice**.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

→ The names of employees and their immediate supervisors. E = employee role, S = supervisor role. Borg, who has no supervisor, is not in the result (7 rows).

Trap: on slide p.32, `FROM EMPLOYEE AS E WHERE E.Super_ssn = E.Ssn` has only one alias, so it becomes a query for "employees who are their own supervisor" (0 rows). It shows that using the same table in two roles requires **two** aliases.



04 p.32 · Aliases (employee–supervisor query using the same table twice)

§ Missing WHERE and *

Missing WHERE: no tuple selection condition → all tuples of the relations in FROM are retrieved.

- With two or more relations, **all possible tuple combinations (the Cartesian product)** are produced
- `SELECT Ssn, Dname FROM EMPLOYEE, DEPARTMENT;` → 8 × 3 = **24 rows**
- Leaving out even one condition gives a wrong and very large result, so **write every condition in WHERE**

Asterisk (*): the values of **all attributes** of the selected tuples. `SELECT * FROM EMPLOYEE WHERE Dno = 5;` → 4 rows, 10 columns.

Two More Features of SQL

❑ Missing WHERE Clause

- No condition on tuple selection
- Retrieve ***all tuples*** of the relations specified in FROM

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

SELECT Ssn
FROM EMPLOYEE;

E.Fname
123456789
333445555
999887777
987654321
666884444
453453453
987987987
888665555

SELECT Ssn, Dname
FROM EMPLOYEE, DEPARTMENT;

Ssn	Dname
123456789	Research
333445555	Research
999887777	Research
987654321	Research
666884444	Research
453453453	Research
987987987	Research
888665555	Research
123456789	Administration
333445555	Administration
999887777	Administration
987654321	Administration
666884444	Administration
453453453	Administration
987987987	Administration
888665555	Administration
123456789	Headquarters
333445555	Headquarters
999887777	Headquarters
987654321	Headquarters
666884444	Headquarters
453453453	Headquarters
987987987	Headquarters
888665555	Headquarters

It is extremely important to specify every condition in the WHERE Clause!
if any such condition is overlooked, incorrect and very large relations may result

All possible tuple combinations
3*8=24

33

04 p.33 · WHERE omitted → 8 × 3 = 24 combinations

Multisets, Set Operations, ORDER BY, Operators

Slides: 4-1 p.35–41

§ Multisets and DISTINCT

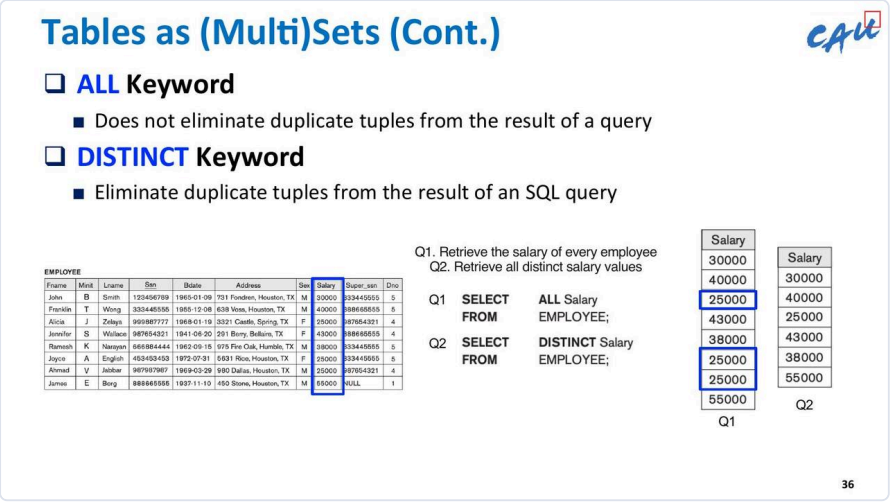
SQL treats a table not as a set but as a multiset → a table or query result may contain **duplicate tuples**, and SQL **does not eliminate duplicates automatically**.

Why?

- Duplicate elimination is an **expensive** operation (sort the tuples first, then remove duplicates)
- The user may want to see duplicates
- Duplicates are needed when applying aggregate functions (e.g., average salary) → week 5

Keywords

- **ALL** (default): keeps duplicates. `SELECT ALL Salary FROM EMPLOYEE;` → 8 rows (25000 appears 3 times)
- **DISTINCT**: eliminates duplicates. `SELECT DISTINCT Salary FROM EMPLOYEE;` → 6 rows



04 p.36 · ALL Salary (8 rows) vs DISTINCT Salary (6 rows)

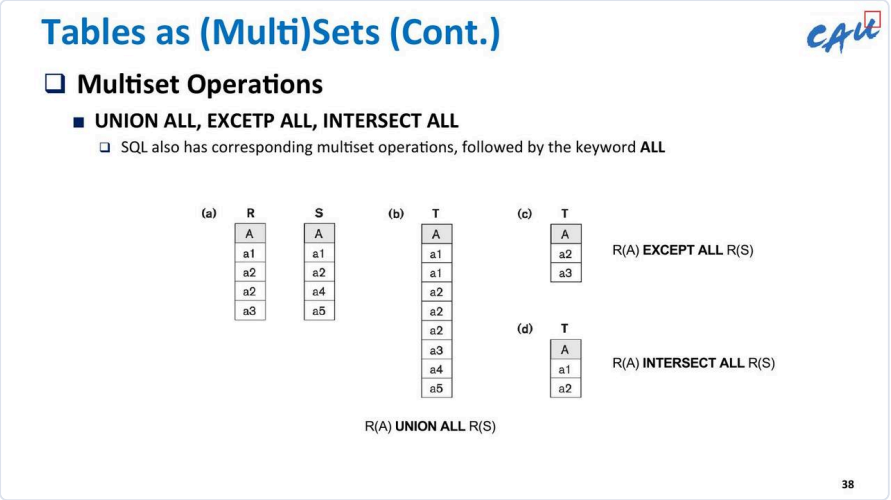
§ Set operations UNION / EXCEPT / INTERSECT

- **UNION**, **EXCEPT** (difference), **INTERSECT**. The result is a **set** of tuples → **duplicates are eliminated**
- Applies only to **type-compatible** relations (same number of attributes, and corresponding attributes have the same domain)
- Multiset versions: **UNION ALL, EXCEPT ALL, INTERSECT ALL** (keep duplicates)
  - R(A) = {a1, a2, a2, a3}, S(A) = {a1, a2, a4, a5}
  - R UNION ALL S = {a1, a1, a2, a2, a2, a3, a4, a5}
  - R EXCEPT ALL S = {a2, a3}
  - R INTERSECT ALL S = {a1, a2}

Query 4: project numbers of projects that involve an employee whose last name is 'Smith', either as a **worker** or as a **manager** of the department that controls the project

```
( SELECT DISTINCT Pnumber
  FROM   PROJECT, DEPARTMENT, EMPLOYEE
  WHERE  Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith' )
UNION
( SELECT DISTINCT Pnumber
  FROM   PROJECT, WORKS_ON, EMPLOYEE
  WHERE  Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith' );
```

→ Smith is not a manager, so the first block is empty; as a worker, 1 and 2 → result {1, 2}



04 p.38 · UNION ALL, EXCEPT ALL, INTERSECT ALL

---

§ ORDER BY, arithmetic operators, BETWEEN

ORDER BY: sorts result tuples by attribute values. **Default is ascending (ASC)**; DESC for descending.

```
SELECT   D.Dname, E.Lname, E.Fname, P.Pname
FROM     DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE    D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno = P.Pnumber
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC;
```

Arithmetic operators + − * /: applied to numeric attributes. **The relation state does not change** (only the result reflects it)

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM   EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P
WHERE  E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'ProductX';
```

→ Smith 33000, English 27500

BETWEEN: a range condition. `(Salary BETWEEN 30000 AND 40000)` = `((Salary ⩾ 30000) AND (Salary ⩽ 40000))` → **both ends inclusive**

```
SELECT * FROM EMPLOYEE WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

→ Smith (30000), Wong (40000), Narayan (38000)

Lab note (3-2, week 6 lab): the lab uses SQLite + Python (pandas, sqlglot). Read NL_queries.txt, write SQL in mysql.py, and run test.py to produce the result CSVs. The submitted zip file must be named with your student ID. **SQLite does not support ALL or ANY.**

U5 · 5-1_Advanced-SQL.pdf

Advanced SQL (Nested Queries, Joins, Aggregation, Views)

Nested queries (IN, ALL/ANY, EXISTS), joined tables (NATURAL, OUTER), aggregation with GROUP BY/HAVING, assertions and triggers, views. This is the core of the SQL-writing questions, so solve lots of problems — especially "... who do ALL ..." queries (division) and GROUP BY + HAVING.

Nested Queries, IN, ALL/ANY

Slides: 5-1 p.3–6

§ Nested queries and IN

Nested query: a complete SELECT-FROM-WHERE block **within the WHERE clause** of another query.

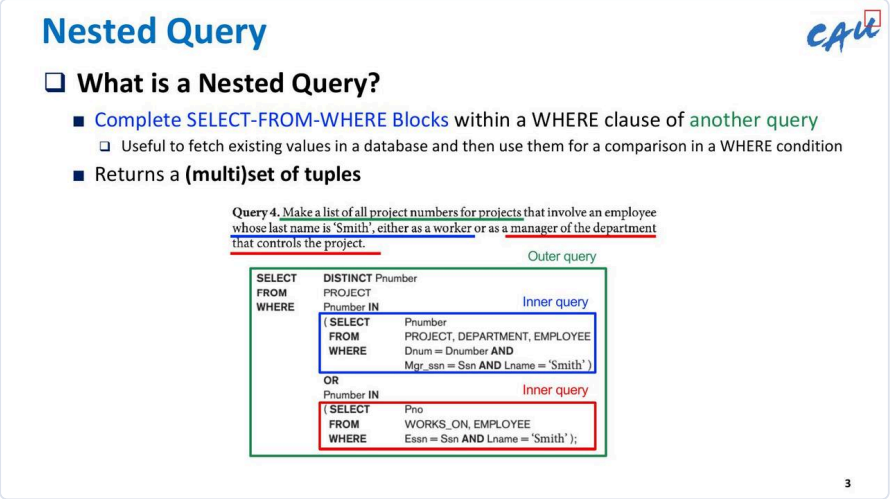
- Useful when we need to fetch existing values from the database and use them in a comparison condition
- The result is a (multi)set of tuples → the enclosing one = outer query, the inner one = inner query

IN: compares a value v with a (multi)set of values V. **TRUE if v is one of the elements in V**.

```
SELECT DISTINCT Pnumber
FROM   PROJECT
WHERE  Pnumber IN ( SELECT Pnumber
                    FROM   PROJECT, DEPARTMENT, EMPLOYEE
                    WHERE  Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith' )
   OR  Pnumber IN ( SELECT Pno
                    FROM   WORKS_ON, EMPLOYEE
                    WHERE  Essn = Ssn AND Lname = 'Smith' );
```

→ Same result as Query 4 (UNION version): {1, 2}. The inner queries V1 and V2 each return a set of project numbers.

> **Lecture note** `Week 5-1`
> The professor said to first think about what form the inner query's result takes: it yields 'a set of Pnumbers', and IN checks whether the current tuple's Pnumber is in that set. The two conditions are joined with OR, so satisfying either one puts the tuple in the result.



05 p.3 · Nested query: two inner queries inside the outer query

§ ALL / ANY (SOME)

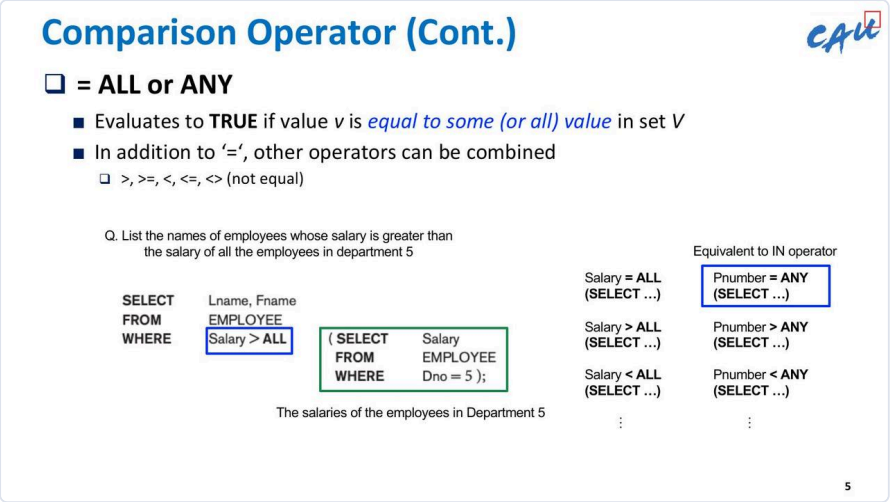
= ANY (= SOME): TRUE if v equals **some** value in V → **equivalent to IN**. **= ALL**: TRUE if v equals **every** value.

- Can be combined with >, >=, <, <=, <> instead of =

```
SELECT Lname, Fname
FROM   EMPLOYEE
WHERE  Salary > ALL ( SELECT Salary FROM EMPLOYEE WHERE Dno = 5 );
```

→ Employees whose salary is greater than that of **all** employees in department 5. Dept 5 salaries = {30000, 40000, 38000, 25000} → above 40000 → Wallace (43000), Borg (55000)

Trap: `> ANY` means "greater than at least one" = greater than the minimum. `> ALL` must be greater than the maximum. Also, the lab's **SQLite does not support ALL/ANY**, so rewrite it as `> (SELECT MAX(Salary) ...)`.



05 p.5 · Salary > ALL (salaries in department 5); = ANY is equivalent to IN

§ Ambiguity in nested queries and correlated queries

With several levels of nesting, if the outer FROM and the inner FROM have **attributes with the same name**, the query becomes ambiguous → resolve it with **aliases**.

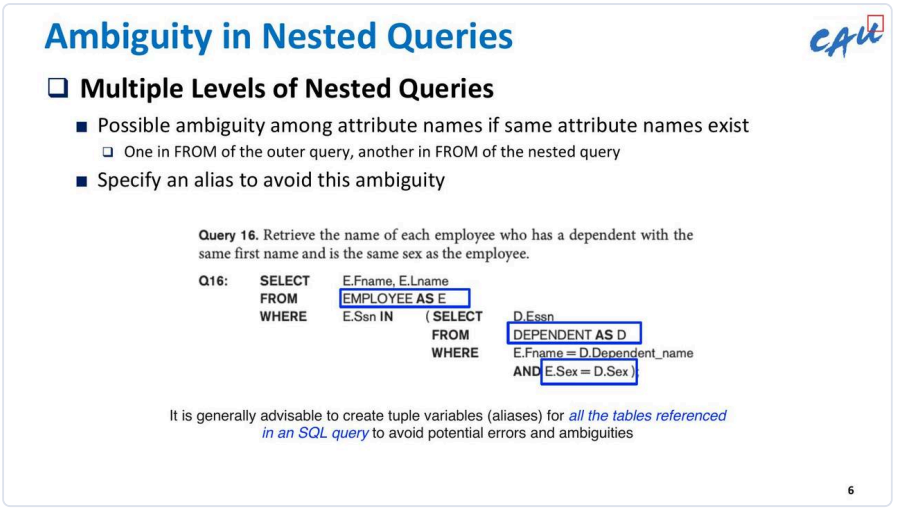
Query 16: employees who have a dependent with the **same first name and the same sex** as the employee

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E
WHERE  E.Ssn IN ( SELECT D.Essn
                  FROM   DEPENDENT AS D
                  WHERE  E.Fname = D.Dependent_name AND E.Sex = D.Sex );
```

- The inner query references the outer E → **correlated nested query**: behaves as if the inner query were evaluated once for each outer tuple
- If you write Sex in the inner query without an alias, it refers to the attribute of the nearest (innermost) table
- On the COMPANY state the result is 0 rows (no such dependent)

Exam point: "It is generally advisable to create tuple variables (aliases) for **all the tables referenced** in a query" (the slide sentence as written).

Lecture note `Week 5-1`
As queries get more complex the conditions get confusing, so the professor stressed 'making it a habit' to give every table an alias (tuple variable) such as E, D, S.



05 p.6 · Query 16: inner query referencing the outer E (ambiguity removed with aliases)

EXISTS / NOT EXISTS, Division Queries

Slides: 5-1 p.7–9

§ EXISTS and NOT EXISTS

Boolean functions that check **whether the result of a nested query is empty**. Usually used in WHERE.

- **EXISTS (Q)**: TRUE if there is **at least one tuple** in the result of Q
- **NOT EXISTS (Q)**: TRUE if the result of Q is **empty**

Query 6: employees who have **no** dependents

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  NOT EXISTS ( SELECT * FROM DEPENDENT WHERE Ssn = Essn );
```

→ Zelaya, Narayan, English, Jabbar, Borg (5 employees)

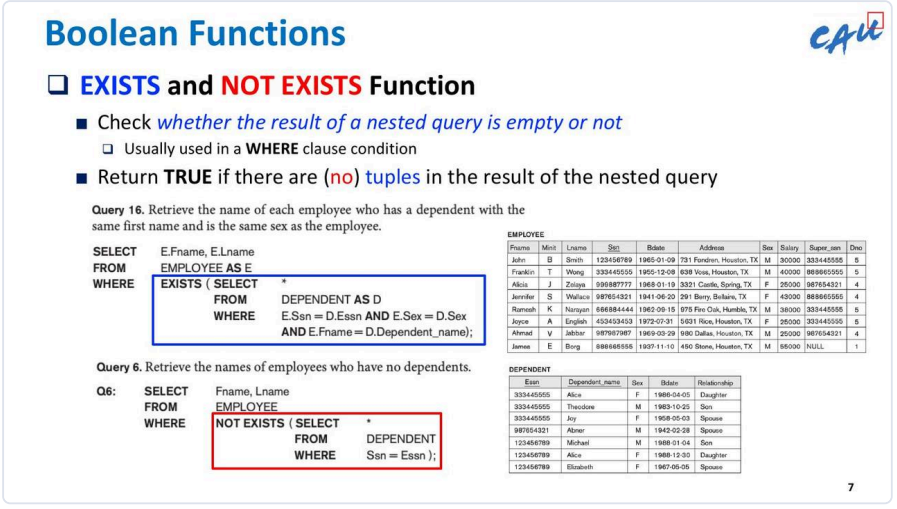
Managers who have at least one dependent: AND the two conditions

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  EXISTS ( SELECT * FROM DEPENDENT  WHERE Ssn = Essn )
  AND  EXISTS ( SELECT * FROM DEPARTMENT WHERE Ssn = Mgr_ssn );
```

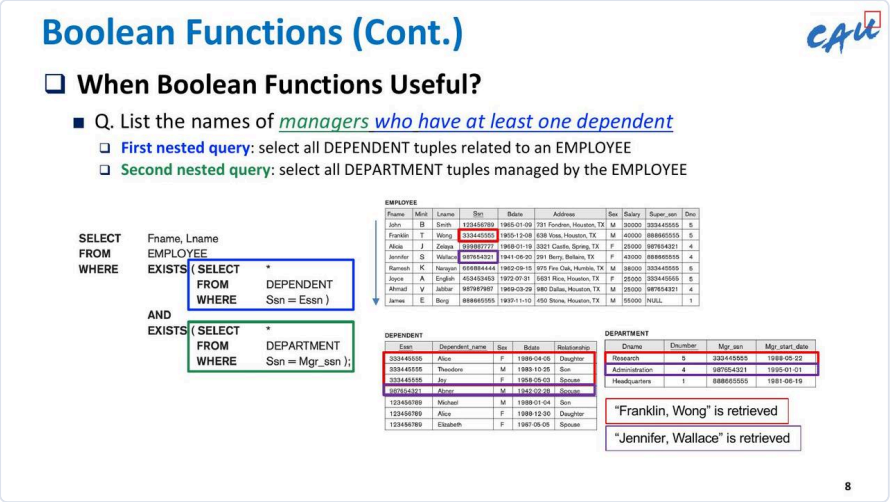
→ Franklin Wong, Jennifer Wallace

Lecture note `Week 5-1`

How to read NOT EXISTS: if the inner query's result is not empty, that employee has dependents. NOT EXISTS is true only when the result is empty, and then the employee appears. For 'managers who have dependents', split the condition into two parts, 'employees who are managers' and 'employees with at least one dependent', and combine two EXISTS with AND.



05 p.7 · EXISTS (Query 16) and NOT EXISTS (Query 6, employees with no dependents)



05 p.8 · Two EXISTS: managers with dependents → Wong, Wallace

§ "For all ..." (division) queries

Exercise 1: names of employees who work on **all** the projects controlled by department 5

```
SELECT Fname, Lname
FROM   EMPLOYEE
WHERE  NOT EXISTS ( ( SELECT Pnumber FROM PROJECT WHERE Dnum = 5 )
                    EXCEPT
                    ( SELECT Pno FROM WORKS_ON WHERE Ssn = Essn ) );
```

How to read it: (all dept-5 projects) − (projects this employee works on) = **missing projects**. If **nothing** is missing (NOT EXISTS), the employee works on all of them.

- e.g., dept-5 projects = {1, 2, 3}. A works on {1, 2, 4} → difference {3} ≠ ∅ → out. B works on {1, 2, 3, 4} → difference ∅ → selected
- On the COMPANY state, dept-5 projects = {1, 2, 3}, but no employee works on all three, so the result is **0 rows**

It can also be written without EXCEPT, using NOT EXISTS twice ("there is no dept-5 project that this employee does not work on"):

```
SELECT E.Fname, E.Lname
FROM   EMPLOYEE AS E
WHERE  NOT EXISTS ( SELECT * FROM PROJECT AS P
                    WHERE  P.Dnum = 5
                      AND  NOT EXISTS ( SELECT * FROM WORKS_ON AS W
                                        WHERE W.Essn = E.Ssn AND W.Pno = P.Pnumber ) );
```

Trap: distinguish employees who work on "some" project (EXISTS, IN) from employees who work on "all" projects (NOT EXISTS + EXCEPT). If the sentence says **all**, it is the division pattern.

Lecture note `Week 5-1`

The professor stressed reading the question slowly: department 5 controls several projects (e.g., 5 to 7), and we need employees who work on 'all' of them, not 'some'. Blackboard example: dept-5 projects = {1, 2, 3}, employee A = {1, 2, 4}, employee B = {1, 2, 3, 4} → {1,2,3} − A = {3} (not empty, rejected), {1,2,3} − B = ∅ (selected). 'NOT EXISTS + EXCEPT' is one solution.

Joined Tables, NATURAL, INNER/OUTER

Slides: 5-1 p.10–14

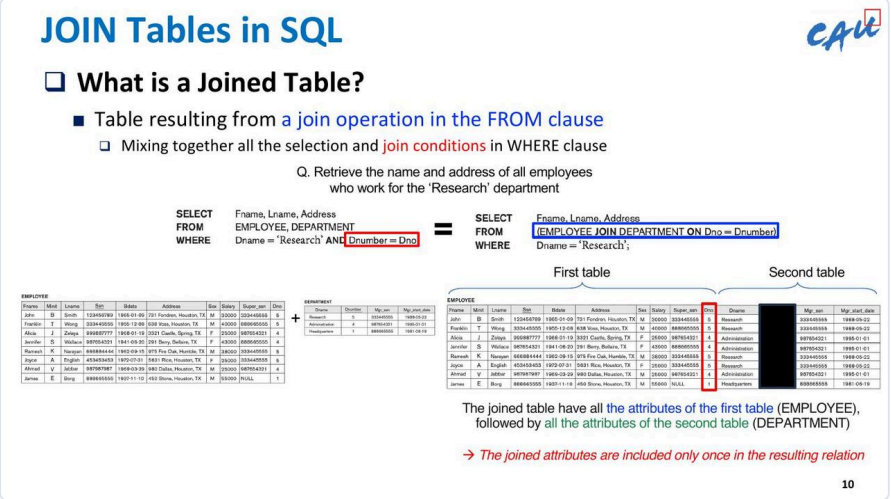
§ Joined tables

Joined table: a table produced by a join operation **in the FROM clause**. It avoids mixing selection and join conditions in WHERE.

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE  Dname = 'Research';
```

- The joined table = all attributes of the first table (EMPLOYEE) + all attributes of the second table (DEPARTMENT)
- Same result as Query 2, which writes `Dnumber = Dno` in WHERE

> **Lecture note** `Week 5-1`
> Dname = 'Research' is the selection condition and Dnumber = Dno is the join condition. The join condition can be written explicitly as JOIN ... ON in the FROM clause, and the result is the same as writing it in WHERE.



05 p.10 · EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber in the FROM clause

§ NATURAL JOIN

Creates an **implicit equality condition for each pair of attributes with the same name** → no join condition is written.

- If the names differ, **rename** the attributes first to make them match

```
SELECT Fname, Lname, Address
FROM   (EMPLOYEE NATURAL JOIN
        (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
WHERE  Dname = 'Research';
```

- DEPARTMENT's Dnumber is renamed to Dno so that it matches EMPLOYEE.Dno
- The join attribute appears **only once** in the result

Lecture note Week 5-1

NATURAL JOIN assumes each pair of attributes with the same name is a join attribute. DEPARTMENT must be aliased as DEPT(Dname, Dno, Mssn, Msdate), renaming Dnumber to Dno, so that the name matches EMPLOYEE.Dno and the natural join works.

§ INNER JOIN and OUTER JOIN

Inner join: the default join. **Only tuples that have a match in the other relation** are included in the result.

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

→ 7 rows. **Borg, whose Super_ssn is NULL, is left out**.

Outer join: tuples without a match are also included, and the other relation's attributes are **padded with NULL**.

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM   (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S ON E.Super_ssn = S.Ssn);
```

→ 8 rows. Borg's Supervisor_name = NULL.

Join	Tuples in the result
INNER JOIN	Only pairs that satisfy the join condition
LEFT OUTER JOIN	**Every tuple of the left table** (right attributes NULL if no match)
RIGHT OUTER JOIN	**Every tuple of the right table**
FULL OUTER JOIN	Every tuple of both tables

Trap: to select "only the unmatched ones" with an OUTER JOIN, add `WHERE B.KEY IS NULL` (bottom row of the Venn diagrams on slide p.14). Compare with NULL using **IS NULL**, not `= NULL`.

Lecture note Week 5-1

In the Super_ssn = Ssn self-join, James Borg has no Super_ssn (NULL), so he does not appear in the inner join result. To also see employees without a supervisor, use LEFT OUTER JOIN. Which join to use depends on the application.

Outer Join

What is Outer Join?

- To retrieve tuples *if it does not have a matching tuple*
  - It is padded with NULL values for the attributes

```
SELECT   E.Lname AS Employee_name,
         S.Lname AS Supervisor_name
FROM     (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
ON E.Super_ssn = S.Ssn);
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Ssn	Super_ssn
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321
888665555	NULL

In LEFT OUTER JOIN,
every tuple in the left table must appear in the result.

13

05 p.13 · LEFT OUTER JOIN: Borg is in the result too (supervisor NULL)

Visualization of Inner and Outer Joins

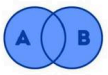
Visualized Different Types of Joins



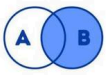
Inner join



SELECT * FROM
A LEFT JOIN B
ON A.KEY = B.KEY



SELECT * FROM
A FULL OUTER JOIN B
ON A.KEY = B.KEY

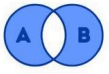


SELECT * FROM
A RIGHT JOIN B
ON A.KEY = B.KEY



SELECT * FROM
A LEFT JOIN B
ON A.KEY = B.KEY
WHERE B.KEY IS NULL

LEFT OUTER JOIN



SELECT * FROM
A FULL OUTER JOIN B
ON A.KEY = B.KEY WHERE A.KEY
IS NULL OR B.KEY IS NULL

FULL OUTER JOIN



SELECT * FROM
A RIGHT JOIN B
ON A.KEY = B.KEY
WHERE A.KEY IS NULL

RIGHT OUTER JOIN

14

05 p.14 · Venn diagrams of inner and outer joins

Aggregate Functions, GROUP BY, HAVING

Slides: 5-1 p.15–22, 48

§ Aggregate functions

Aggregate function: summarizes information from multiple tuples into a **single-tuple summary**. Mostly used in SELECT or HAVING.

- **COUNT**: the number of tuples or values
- **SUM, MAX, MIN, AVG**: applied to a (multi)set of numeric values

Query 19: sum, maximum, minimum and average salary of all employees

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM   EMPLOYEE;
```

→ 281000, 55000, 25000, 35125

Query 20: the Research department only

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM   (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE  Dname = 'Research';
```

→ 133000, 40000, 25000, 33250

> **Lecture note** Week 5-1
> Without aggregate functions you would have to read all the salaries and compute the sum, maximum and average yourself with for/while loops in Python or C. Aggregate functions do it in one line. Results: maximum 55,000, minimum 25,000, average = sum ÷ 8.

§ COUNT and NULL, aggregates in nested queries

- **COUNT(*)**: the **number of rows (tuples)** in the result. Query 21 `SELECT COUNT(*) FROM EMPLOYEE;` → 8; Query 22, Research only → 4
- **COUNT(attribute)**: the number of **values** of that attribute. **NULLs are not counted**. e.g., `COUNT(Super_ssn)` → 7
- **COUNT(DISTINCT Salary)**: the number of distinct salary values → 6 (Query 23)
- An aggregate result is a single value, so it can be used as **part of a condition**:

```
SELECT Lname, Fname
FROM   EMPLOYEE
WHERE  ( SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn ) ⩾ 2;
```

→ Two or more dependents: Smith (3), Wong (3)

Lecture note Week 5-1

Without COUNT(*) the 8 tuples come out as they are, but with COUNT(*) only a single number (8) comes out. Research gives 4. The number of distinct salary values is 6 because 25,000 repeats. An aggregate result is a single value, so it can be used as part of a condition such as 'two or more dependents': the inner query returns one dependent count per employee, which is compared with 2.

§ GROUP BY

Grouping: partitions the tuples of a relation into **nonoverlapping subsets (groups)** and applies aggregate functions to each group.

- **Grouping attribute**: the basis for forming groups. Same group = tuples with the same value of the grouping attribute(s)
- Specified in the **GROUP BY** clause. The grouping attributes should also appear in SELECT so the result shows which group each row is

Query 24: for each department, the department number, number of employees and average salary

```
SELECT   Dno, COUNT(*), AVG(Salary)
FROM     EMPLOYEE
GROUP BY Dno;
```

→ (5, 4, 33250), (4, 3, 31000), (1, 1, 55000)

Query 25: for each project, its number, name and number of employees who work on it

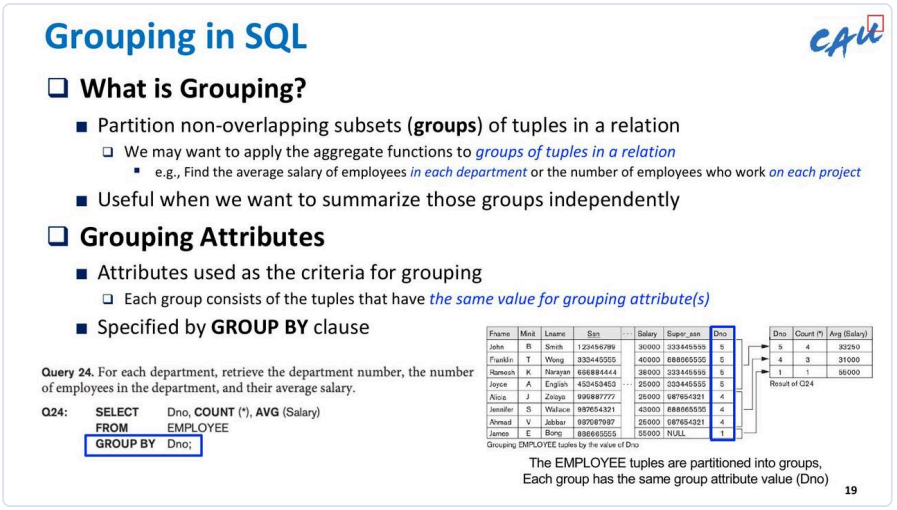
```
SELECT   Pnumber, Pname, COUNT(*)
FROM     PROJECT, WORKS_ON
WHERE    Pnumber = Pno
GROUP BY Pnumber, Pname;
```

→ ProductX 2, ProductY 3, ProductZ 2, Computerization 3, Reorganization 3, Newbenefits 3

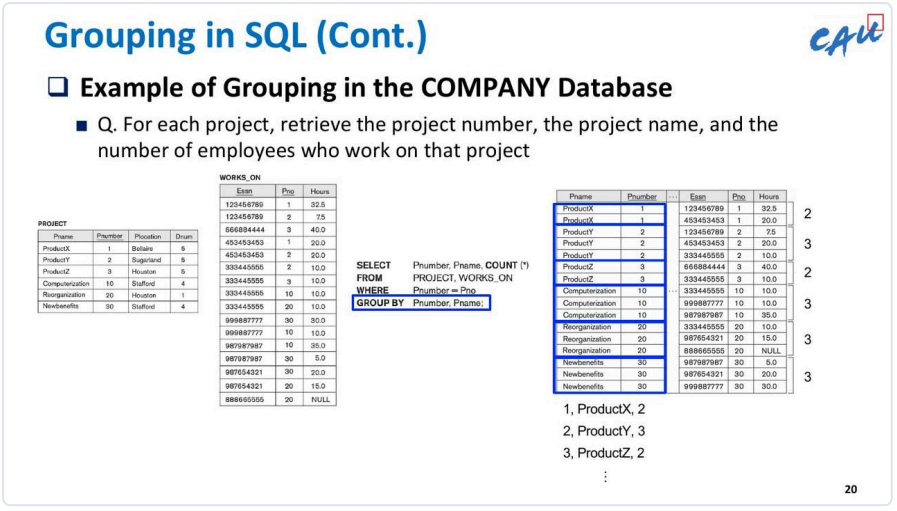
Trap: with GROUP BY, SELECT may contain **only grouping attributes and aggregate functions**. Do not put an attribute that does not have a single value per group (e.g., Lname when grouping by Dno) in SELECT.

Lecture note Week 5-1

Grouping is used to summarize each group separately. Grouping by Dno creates three groups, departments 5, 4 and 1, and COUNT and AVG are computed for each group.



05 p.19 · Query 24: grouping by Dno



05 p.20 · Query 25: number of employees per project

§ HAVING

HAVING: a **condition on the summary information** of groups. Used with GROUP BY. **Only groups** that satisfy the condition appear in the result.

Query 26: for each project on which **more than two** employees work, the project number, name and number of employees

```
SELECT   Pnumber, Pname, COUNT(*)
FROM     PROJECT, WORKS_ON
WHERE    Pnumber = Pno
GROUP BY Pnumber, Pname
HAVING   COUNT(*) > 2;
```

→ ProductY, Computerization, Reorganization, Newbenefits (3 employees each)

Exercise 2: for each department whose average salary is greater than $30,000, the department name and number of employees

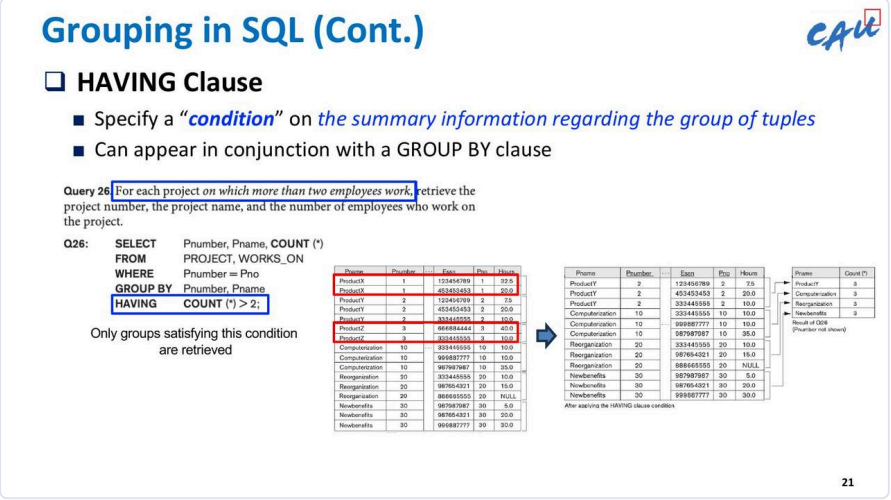
```
SELECT   D.Dname, COUNT(*)
FROM     EMPLOYEE AS E, DEPARTMENT AS D
WHERE    E.Dno = D.Dnumber
GROUP BY D.Dname
HAVING   AVG(E.Salary) > 30000;
```

→ Research 4, Administration 3, Headquarters 1 (the slide solution groups by E.Dno; in standard SQL it is safer to also have D.Dname from SELECT in GROUP BY)

> Exam point: **WHERE = tuple condition (before groups are formed), HAVING = group condition (after aggregation)**. A condition on an aggregate, such as "average above 30,000", cannot go in WHERE.

Order of execution: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

> **Lecture note** `Week 5-1`
> GROUP BY summarizes per group; HAVING summarizes only the groups that satisfy a condition. While solving Exercise 2 the professor asked 'why do we need DEPARTMENT and not just EMPLOYEE?' → EMPLOYEE has only the department number, not the department 'name', so a join is needed. The average condition (> 30000) goes in HAVING.



05 p.21 · Query 26: HAVING COUNT(*) > 2

Assertions, Triggers

Slides: 5-1 p.24–29

§ Assertions

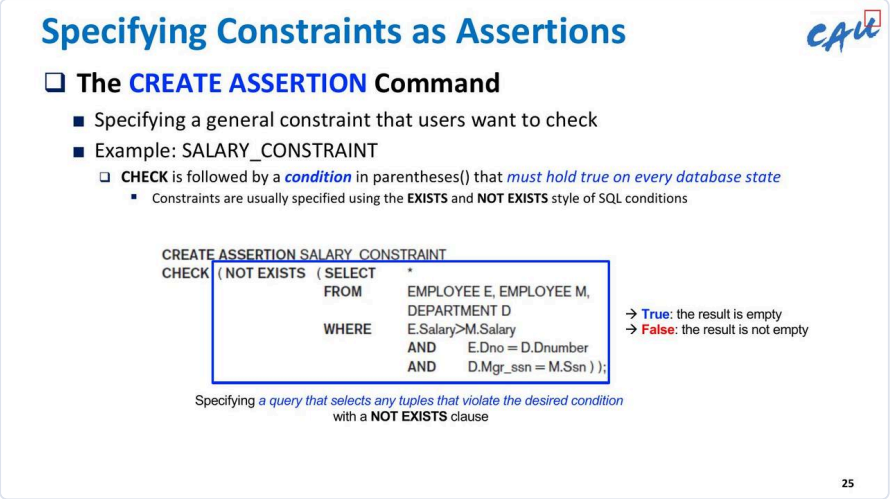
Specify **general constraints that cannot be expressed by the built-in constraints (key, entity integrity, referential integrity)**.

- e.g., the salary of an employee must not be greater than the salary of the manager of the department the employee works for

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM   EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
                     WHERE  E.Salary > M.Salary
                       AND  E.Dno = D.Dnumber
                       AND  D.Mgr_ssn = M.Ssn ) );
```

- The condition in parentheses after **CHECK** must be **true in every database state**
- Convention: write a **query that selects the tuples violating** the desired condition and wrap it in **NOT EXISTS** → if the result is empty, TRUE (satisfied); if not empty, FALSE (violated)

> **Lecture note** `Week 5-1`
> CREATE ASSERTION name CHECK (condition). The convention is to write 'a query that finds the tuples violating the condition you want to check' and wrap it in NOT EXISTS. If an employee's salary is greater than the manager's salary it is a violation → if that query's result is not empty, there is a problem.



05 p.25 · CREATE ASSERTION SALARY_CONSTRAINT (violation query wrapped in NOT EXISTS)

§ Triggers (ECA rules)

Specify **actions to be executed automatically when certain events occur and conditions are met**. **ECA (Event, Condition, Action) rules**.

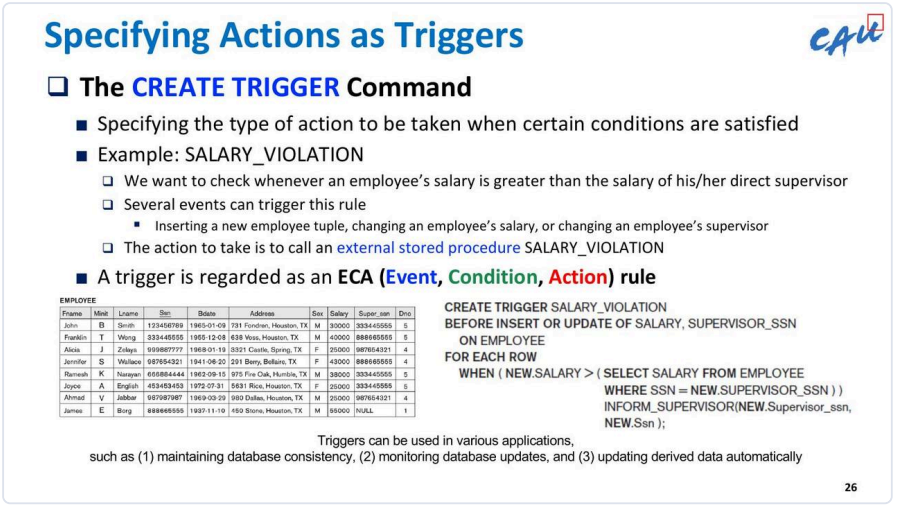
```
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
    ON EMPLOYEE
FOR EACH ROW
    WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                          WHERE SSN = NEW.SUPERVISOR_SSN ) )
        INFORM_SUPERVISOR(NEW.Supervisor_ssn, NEW.Ssn);
```

- **Event**: database update operations explicitly applied to the DB (inserting a new employee, changing a salary, changing a supervisor). **BEFORE / AFTER** runs the trigger before/after the operation. **All** possible events must be considered
- **Condition**: determines whether the action should be executed after the event occurs. The **WHEN clause**. Optional. The action runs only when it is true
- **Action**: what is executed automatically = a sequence of SQL statements, a database transaction, or an external program (e.g., the external stored procedure INFORM_SUPERVISOR)
- Uses: (1) maintaining database consistency, (2) monitoring database updates, (3) updating derived data automatically
- e.g., notify the manager when an employee's travel expenses exceed a limit

Trap: Assertion = a **constraint** (blocks the operation if violated); Trigger = an **action** (does something when the condition holds). Both slide examples compare salaries, but one "blocks" and the other "notifies".

Lecture note `Week 5-1`

A trigger = three components: Event, Condition, Action. The Event is an update operation that could break the condition (inserting a new employee, changing a salary, changing a supervisor), with BEFORE/AFTER specifying the timing. The Condition is the WHEN clause; if it is not true, the action is skipped. The Action can be anything: SQL statements, a transaction, or an external program (e.g., notifying the supervisor via a messenger).



05 p.26 · CREATE TRIGGER SALARY_VIOLATION (ECA)

Views

Slides: 5-1 p.30–47

§ Definition and use of views

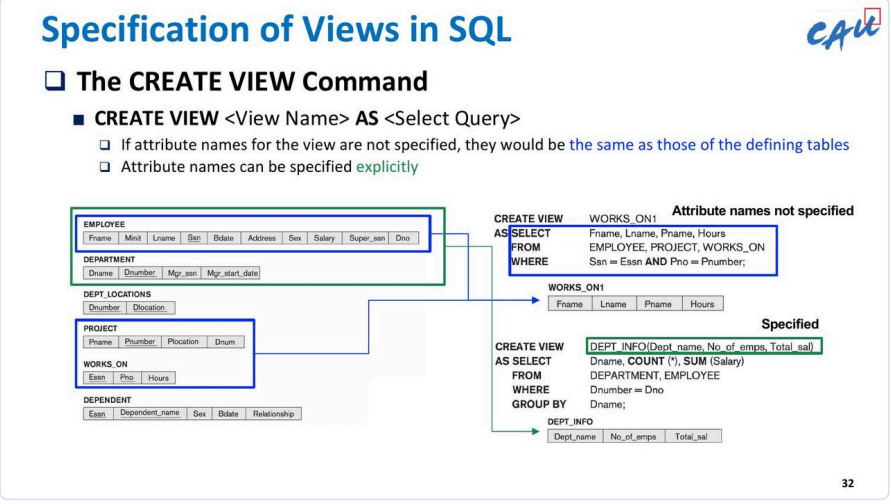
View: a **virtual table** derived from base tables. It may not exist physically.

- Retrieval: no restrictions. Update: usually limited (effectively read-only)
- Why use it: **simplifies complex queries**. If you often use a multi-table join or aggregation, define it as a view and use it like a **single-table query**

```
CREATE VIEW WORKS_ON1
AS SELECT Fname, Lname, Pname, Hours
   FROM   EMPLOYEE, PROJECT, WORKS_ON
   WHERE  Ssn = Essn AND Pno = Pnumber;

CREATE VIEW DEPT_INFO (Dept_name, No_of_emps, Total_sal)
AS SELECT   Dname, COUNT(*), SUM(Salary)
   FROM     DEPARTMENT, EMPLOYEE
   WHERE    Dnumber = Dno
   GROUP BY Dname;
```

- If no attribute names are given, the view uses the attribute names of the defining query (WORKS_ON1); they can also be given explicitly (DEPT_INFO)
- Querying a view: `SELECT Fname, Lname FROM WORKS_ON1 WHERE Pname = 'ProductX';` → no need to join three tables every time
- **DROP VIEW WORKS_ON1;** → the defining tables are not affected



05 p.32 · CREATE VIEW WORKS_ON1, DEPT_INFO

§ View implementation: query modification vs materialization

A view must always be **up-to-date** → when the defining tables change, the DBMS is responsible for reflecting it in the view.

Approach	Description	Characteristics
Query modification	**Transforms the view query into a query on the base tables**. The view is not stored physically	Convenient for users but inefficient for the DB system (complex views are recomputed every time)
View materialization	**Physically creates a temporary view table** when the view is first queried and keeps it for a while	Assumes more queries on the view will follow. **Extra storage space**. When defining tables change, use **incremental update**

When to update a materialized view:

- **Immediate update**: as soon as the base tables change. Always up to date, but sometimes unnecessary and inefficient
- **Periodic update**: periodically. A view query may get a result that is not up to date
- **Lazy update**: when a view query needs it. Not up to date until a query arrives

If a materialized view is not queried for a certain period, it is removed and recomputed from scratch when needed later.

§ Updating views

Retrieval from a view is always possible, but INSERT/DELETE/UPDATE is impossible in many cases.

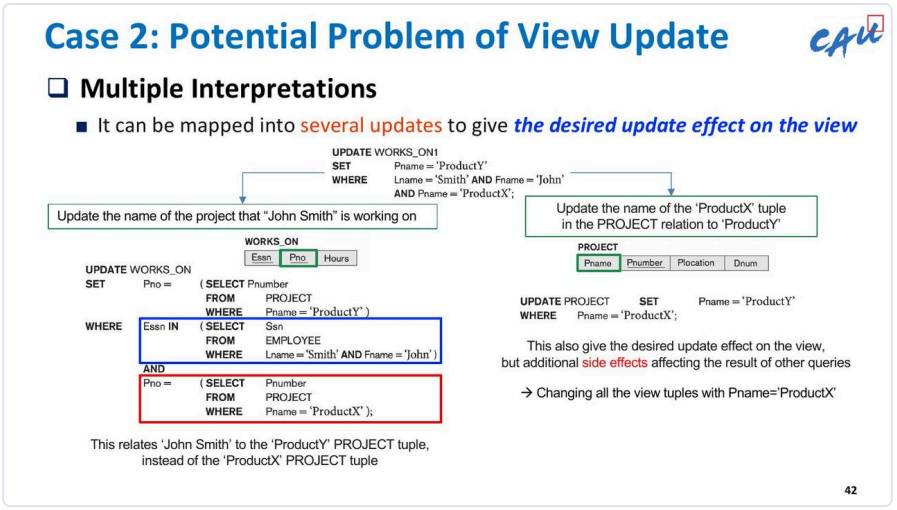
A view is updatable when the view update corresponds to exactly one update on the base relations.

- **Single table + no aggregates** → maps to one base-table update → **allowed**. But the view's attributes must include (1) the base relation's **PK** and (2) **all NOT NULL attributes** without a DEFAULT (when inserting a tuple through the view those values cannot be empty)
- **Join (multiple tables)** → can be mapped in several ways, so **generally not updatable**
- **Grouping and aggregation** → the update **does not make sense** → **not updatable**

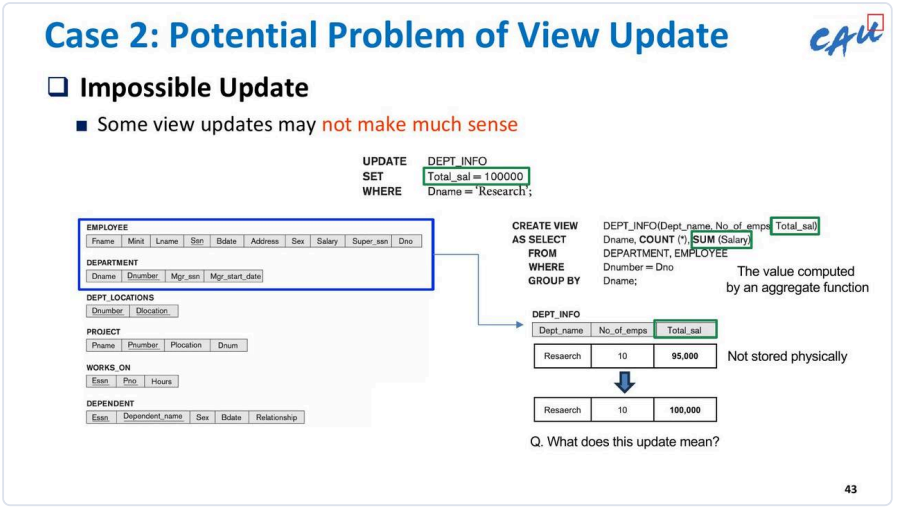
Example 1 (multiple interpretations): `UPDATE WORKS_ON1 SET Pname = 'ProductY' WHERE Lname = 'Smith' AND Fname = 'John' AND Pname = 'ProductX';`

- Interpretation a: change the project Smith works on → set Pno in WORKS_ON to ProductY's number (as intended)
- Interpretation b: rename 'ProductX' in PROJECT to 'ProductY' → the view result looks the same, but with a **side effect**: every tuple that worked on ProductX changes

Example 2 (impossible): `UPDATE DEPT_INFO SET Total_sal = 100000 WHERE Dname = 'Research';` → Total_sal is a **computed value** (SUM), not physically stored → meaningless



05 p.42 · Two interpretations of an update on WORKS_ON1



05 p.43 · Updating Total_sal of DEPT_INFO is meaningless

§ Views for authorization

Views can **hide certain attributes or tuples** from unauthorized users.

```
CREATE VIEW DEPT5EMP AS
SELECT * FROM EMPLOYEE WHERE Dno = 5;            -- restricts tuples

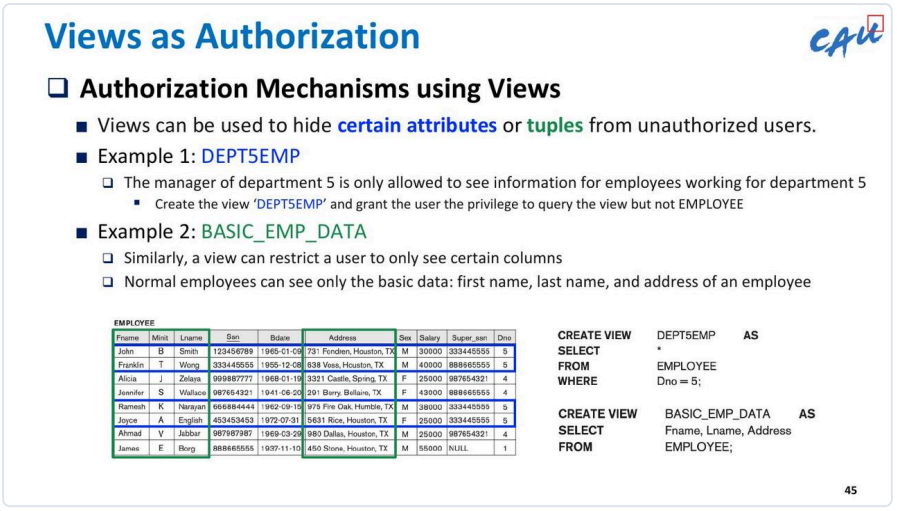
CREATE VIEW BASIC_EMP_DATA AS
SELECT Fname, Lname, Address FROM EMPLOYEE;      -- restricts attributes
```

- DEPT5EMP: lets the manager of department 5 see only department 5 employees. The user is granted **query privileges only on the view**, not on EMPLOYEE
- BASIC_EMP_DATA: general employees see only names and addresses

Slide Exercises 3 and 4, solutions:

```
CREATE VIEW DEPT_INFO (DeptName, MgrFname, MgrLname, MgrSalary)
AS SELECT D.Dname, E.Fname, E.Lname, E.Salary
   FROM   DEPARTMENT AS D, EMPLOYEE AS E
   WHERE  D.Mgr_ssn = E.Ssn;                        -- department and manager info

CREATE VIEW ProjectInfo (Pname, Dname, NumEmployees, TotalHours)
AS SELECT   P.Pname, D.Dname, COUNT(*), SUM(W.Hours)
   FROM     PROJECT AS P, WORKS_ON AS W, DEPARTMENT AS D
   WHERE    P.Pnumber = W.Pno AND D.Dnumber = P.Dnum
   GROUP BY P.Pnumber, P.Pname, D.Dname;          -- project summary
```



05 p.45 · Views for authorization: DEPT5EMP (restricts tuples), BASIC_EMP_DATA (restricts attributes)

U6 · 6-1_relational-algebra.pdf + db-exercises Week 7

Relational Algebra

σ π ρ, set operations, ×, ⋈ (theta, equi, natural), ÷, ℑ (aggregation), outer joins. The key skill is breaking a query into steps of the form "intermediate relation name ← operation". In your answers you may write words such as SELECT, PROJECT, JOIN instead of the symbols and still get credit.

Introduction to Relational Algebra, Query Trees

Slides: 6-1 p.3–6

§ What is relational algebra?

- **Relational algebra**: a theoretical, mathematical model of "**how** to retrieve data" → **procedural**
- **SQL** is a high-level **declarative** language → it states only "**what** data is needed", without caring how it is retrieved
- The **basic set of operations** of the formal relational model. Rigorously based on mathematical concepts of sets

Why it matters:

- The **formal foundation** for relational model operations (select, project, rename, join, Cartesian product, ...)
- **The basis for query processing and optimization**: it specifies the **sequence** of operations that produces the result

> Exam point: "SQL = what (declarative), relational algebra = how (procedural)". Also, relational algebra is a **closed system**: the result of every operation is again a relation.

§ Query (execution) tree

Query tree: a tree used inside the DBMS to represent a query = a relational algebra expression.

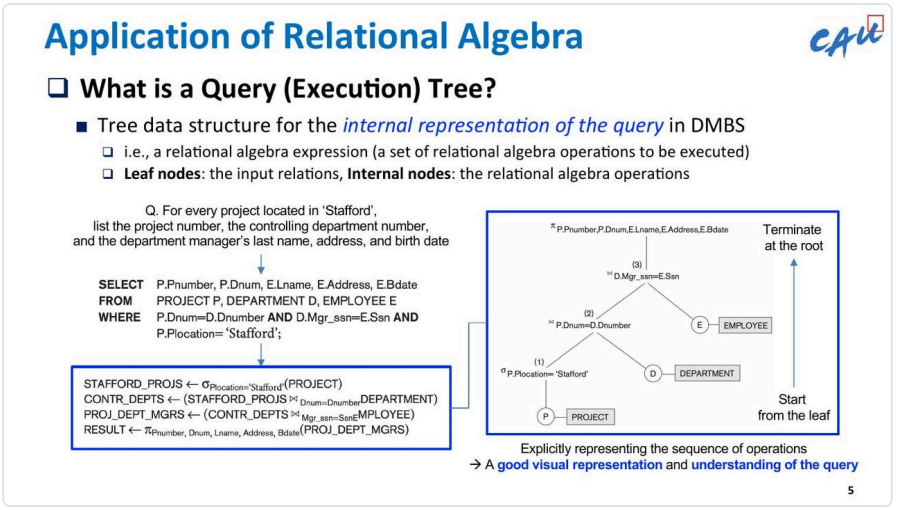
- **Leaf nodes = input relations**, **internal nodes = relational algebra operations**
- **Execution starts at the leaves and ends at the root** (executed bottom-up)
- It shows the order of operations explicitly, which makes the query easy to understand

Slide example: for every project located in 'Stafford', the project number, the controlling department number, and the department manager's last name, address and birth date

```
SELECT P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate
FROM   PROJECT P, DEPARTMENT D, EMPLOYEE E
WHERE  P.Dnum = D.Dnumber AND D.Mgr_ssn = E.Ssn AND P.Plocation = 'Stafford';
```

Tree: (1) σ Plocation='Stafford' (P) → (2) ⋈ P.Dnum=D.Dnumber D → (3) ⋈ D.Mgr_ssn=E.Ssn E → root π Pnumber, Dnum, Lname, Address, Bdate

Query processing: the process of choosing a suitable execution strategy (query tree) among the possible ones. **One SQL query can have several query trees (plans)** → which one is best is query optimization (week 13).



06 p.5 · Query tree for the 'Stafford' query (leaves = relations, internal nodes = operations)

Unary Operations: SELECT, PROJECT, RENAME

Slides: 6-1 p.8–15

§ SELECT (σ)

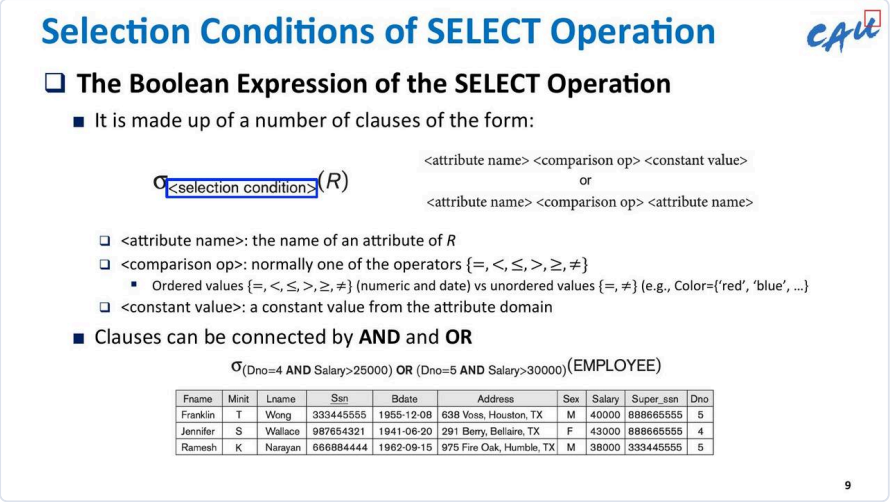
σ<selection condition>(R): selects the **subset of tuples of a relation that satisfy the selection condition → horizontal filtering**.

- The condition is applied to **each tuple individually**
- The result has the **same attributes** as R → **degree of the result = degree of R**
- Condition = clauses of the form `<attribute> <comparison op> <constant>` or `<attribute> <comparison op> <attribute>`, connected by **AND, OR**
  - Comparison operators {=, <, ≤, >, ≥, ≠}. All of them for ordered values (numbers, dates); only {=, ≠} for unordered values (e.g., colors)
- e.g., σ Dno=4 (EMPLOYEE) → Zelaya, Wallace, Jabbar
- e.g., σ (Dno=4 AND Salary>25000) OR (Dno=5 AND Salary>30000) (EMPLOYEE) → Wong, Wallace, Narayan

Properties

- **Commutative**: σ c1(σ c2(R)) = σ c2(σ c1(R))
- A cascade of σ operations **can be combined into one** with AND: σ c1(σ c2(...σ cn(R))) = σ c1 AND c2 AND ... AND cn(R)

Trap: the relational algebra SELECT (σ) corresponds to SQL's **WHERE clause, not the SELECT clause**. `σ Dno=4 AND Salary>25000 (EMPLOYEE)` = `SELECT * FROM EMPLOYEE WHERE Dno=4 AND Salary>25000;`



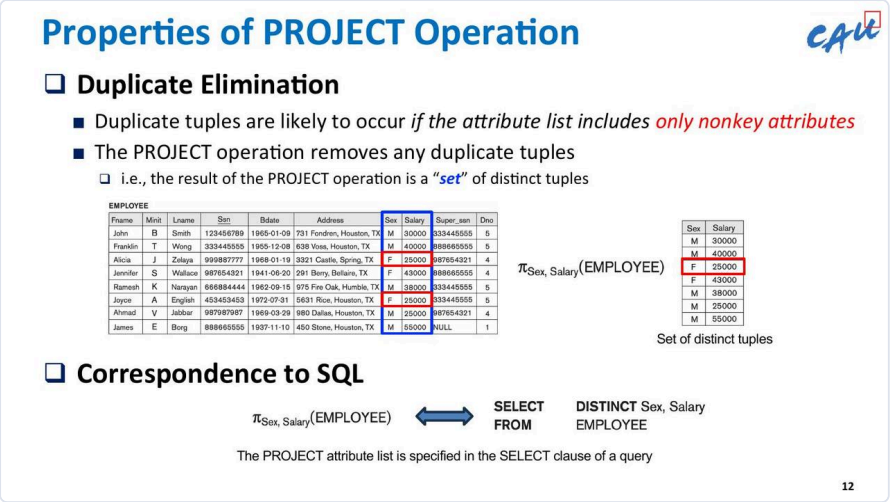
06 p.9 · σ with selection conditions connected by AND/OR

§ PROJECT (π)

π<attribute list>(R): keeps **only the desired columns** and discards the rest → **vertical filtering**.

- Degree of the result = number of attributes in the attribute list

- **Duplicate elimination**: if the attribute list contains no key (only nonkey attributes), duplicate tuples may appear, but π **removes duplicates** → the result is a **set** of distinct tuples
  - e.g., π Sex, Salary (EMPLOYEE) → **7 rows**, not 8 ((F, 25000) appears twice → once)
- SQL counterpart: the π attribute list = the **SELECT clause**. But SQL does not remove duplicates, so the exact equivalent is `SELECT DISTINCT Sex, Salary FROM EMPLOYEE;`

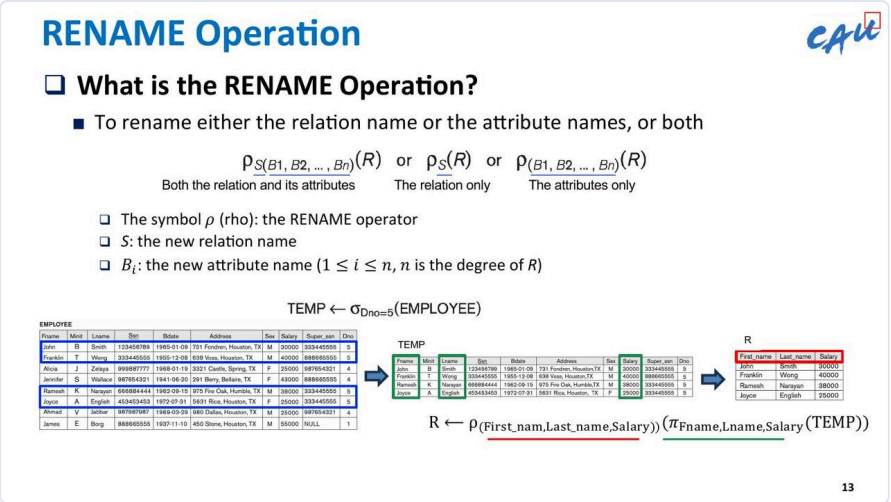


06 p.12 · π Sex, Salary: duplicates removed → SQL's SELECT DISTINCT

§ RENAME (ρ)

ρ: renames the relation, its attributes, or both.

- ρ $S(B_1, ..., B_n)(R)$: both / ρ $S(R)$: relation name only / ρ $(B_1, ..., B_n)(R)$: attribute names only
- Also written as an assignment: `R(First_name, Last_name, Salary) ← π Fname, Lname, Salary (TEMP)`
- In SQL, renaming is done with **AS (aliasing)**



06 p.13 · Three forms of RENAME ρ

§ Sequences of operations (intermediate relations)

Two ways to apply several operations:

- **Method 1**: name the intermediate relations to make the sequence explicit

```
DEP5_EMPS ← σ Dno=5 (EMPLOYEE)
RESULT    ← π Fname, Lname, Salary (DEP5_EMPS)
```

- **Method 2**: **nest** the operations into a single expression

```
π Fname, Lname, Salary ( σ Dno=5 (EMPLOYEE) )
```

→ Smith 30000, Wong 40000, Narayan 38000, English 25000

For complex queries it is **usually easier to break them into intermediate result relations** than to write one expression.

Set Operations, Cartesian Product

Slides: 6-1 p.17–22

§ Union, intersection, difference

Union R ∪ S, **intersection R ∩ S**, **set difference R – S** (MINUS, EXCEPT). **Duplicates are eliminated** in the result.

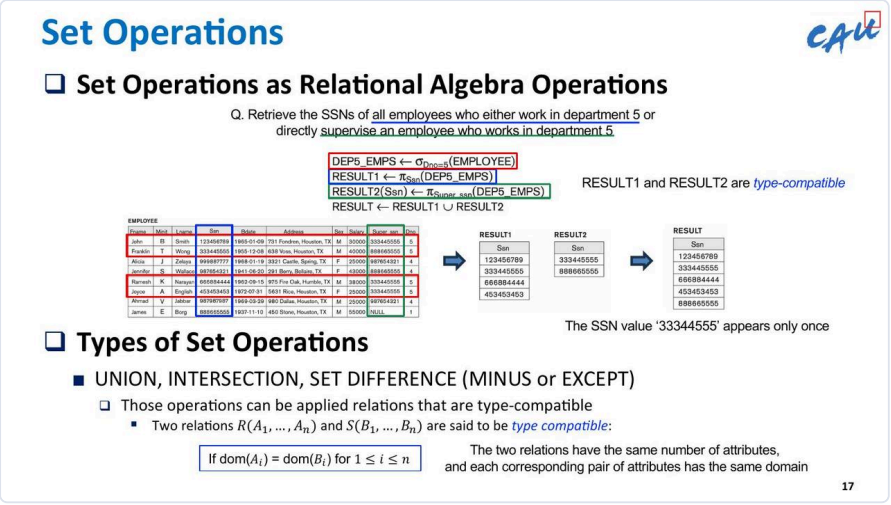
- Applies only to **type compatible (union compatible)** relations: R(A_1...A_n) and S(B_1...B_n) have **the same number of attributes**, and **dom(A_i) = dom(B_i)** for all i
- Slide example: SSNs of all employees who either work in department 5 or directly supervise an employee who works in department 5

```
DEP5_EMPS ← σ Dno=5 (EMPLOYEE)
RESULT1   ← π Ssn (DEP5_EMPS)
RESULT2(Ssn) ← π Super_ssn (DEP5_EMPS)
RESULT    ← RESULT1 ∪ RESULT2
```

→ RESULT1 = {123456789, 333445555, 666884444, 453453453}, RESULT2 = {333445555, 888665555}, RESULT = 5 SSNs (333445555 only once)

Properties

- ∪ and ∩ are **commutative** (R ∪ S = S ∪ R) and **associative** (R ∪ (S ∪ T) = (R ∪ S) ∪ T)
- **– is not commutative**: R – S ≠ S – R (STUDENT – INSTRUCTOR differs from INSTRUCTOR – STUDENT)
- By convention, the result's attribute names are those of the **first relation**



06 p.17 · RESULT1 ∪ RESULT2 (type compatible)

§ Cartesian product (×)

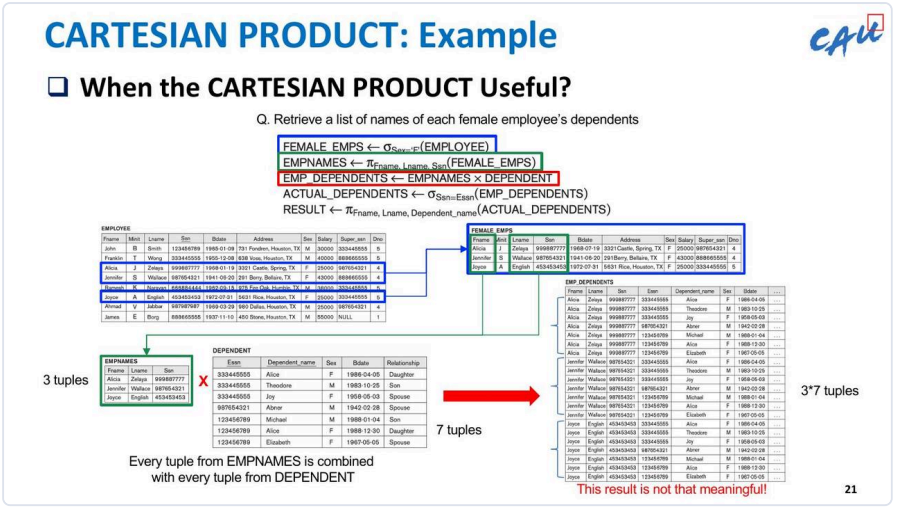
R(A_1...A_n) × S(B_1...B_m) → Q(A_1...A_n, B_1...B_m): concatenates **every possible combination** of tuples from the two relations. Also called CROSS PRODUCT or CROSS JOIN.

- A binary set operation that applies **even when the relations are not type compatible**
- Number of attributes in the result = **n + m**, number of tuples = **|R| × |S|**
- Usually meaningless by itself → useful when **followed by a SELECT** that keeps only related tuples

Slide example: names of the dependents of female employees

```
FEMALE_EMPS       ← σ Sex='F' (EMPLOYEE)
EMPNAMES          ← π Fname, Lname, Ssn (FEMALE_EMPS)
EMP_DEPENDENTS    ← EMPNAMES × DEPENDENT              -- 3 × 7 = 21 rows (meaningless)
ACTUAL_DEPENDENTS ← σ Ssn=Essn (EMP_DEPENDENTS)
RESULT            ← π Fname, Lname, Dependent_name (ACTUAL_DEPENDENTS)
```

→ One row: Jennifer Wallace – Abner. **× followed by σ = JOIN** (next lesson).



06 p.21 · EMPNAMES × DEPENDENT = 3 × 7 = 21 rows

JOIN, DIVISION, Complete Set

Slides: 6-1 p.24–34

§ JOIN (⋈)

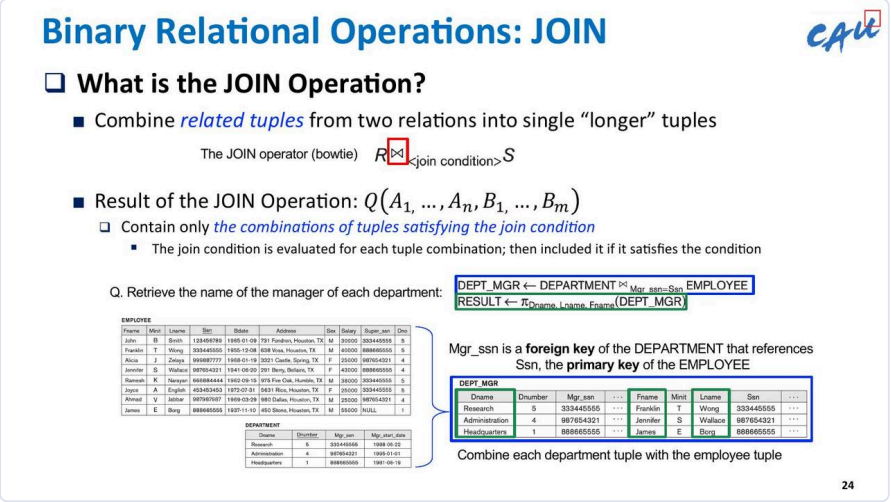
R ⋈<join condition> S: combines **related tuples from two relations into single longer tuples**. The result Q($A_1 \ldots A_n$, $B_1 \ldots B_m$) contains **only the combinations that satisfy the join condition**.

- e.g., the name of the manager of each department

```
DEPT_MGR ← DEPARTMENT ⋈ Mgr_ssn=Ssn EMPLOYEE
RESULT   ← π Dname, Lname, Fname (DEPT_MGR)
```

→ Research–Wong, Administration–Wallace, Headquarters–Borg

- **JOIN = Cartesian product + SELECT**: the two lines `EMP_DEPENDENTS ← EMPNAMES × DEPENDENT; ACTUAL ← σ Ssn=Essn (EMP_DEPENDENTS)` become the one line `ACTUAL ← EMPNAMES ⋈ Ssn=Essn DEPENDENT`

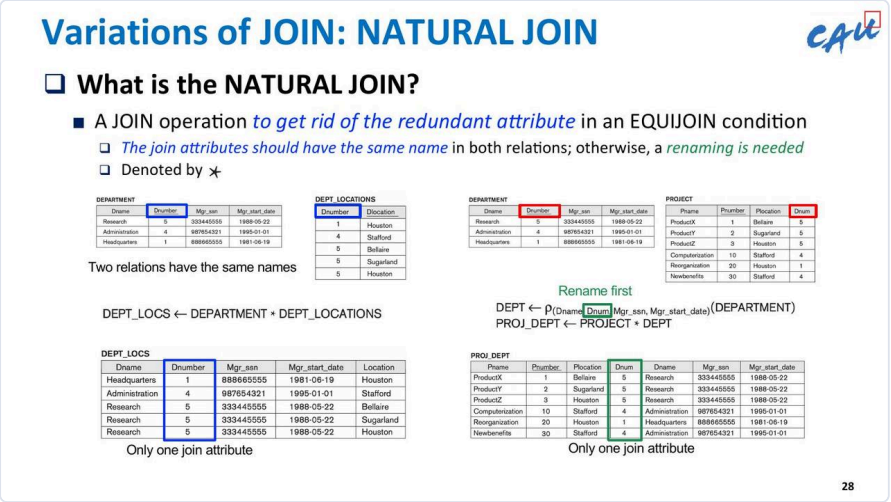


06 p.24 · DEPARTMENT ⋈ Mgr_ssn=Ssn EMPLOYEE

§ Theta, equi and natural joins

- **THETA JOIN**: a general join condition. `<condition> AND <condition> AND …`, each condition of the form $\mathbf{A_i}\ \theta\ \mathbf{B_j}$ ($\theta \in \{=, <, \leq, >, \geq, \neq\}$). Only tuples satisfying all conditions
- **EQUIJOIN**: a join that uses **only =** as the comparison operator. The result has pairs of attributes with identical values (e.g., Mgr_ssn and Ssn) → one of each pair is superfluous
- **NATURAL JOIN (denoted *)**: an equijoin with **one of each pair of duplicate attributes removed**
  - The join attributes must **have the same name** in both relations. If not, **rename them first**
  - `DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS` (Dnumber has the same name) → Dnumber appears once
  - `DEPT ← ρ (Dname, Dnum, Mgr_ssn, Mgr_start_date) (DEPARTMENT); PROJ_DEPT ← PROJECT * DEPT` (matched on Dnum)

Trap: a natural join uses **all** attributes with the same name as join conditions. If some attributes accidentally share a name (e.g., both have Name), an unintended condition is added.



06 p.28 · Natural join (*): on attributes with the same name; if names differ, RENAME first

§ Size of a join result

- **0 ≤ |R ⋈ S| ≤ |R| × |S|**
- Minimum: if no combination satisfies the join condition → an **empty relation**
- Maximum: if every combination satisfies the condition, or there is no condition → same as the **Cartesian product**
- **Join selectivity** = expected size of the join result / (|R| × |S|). Used in query optimization to estimate the cost of query plans

§ DIVISION (÷)

An operation for "... who do **all** ..." queries. It can be expressed with basic operations but is defined separately.

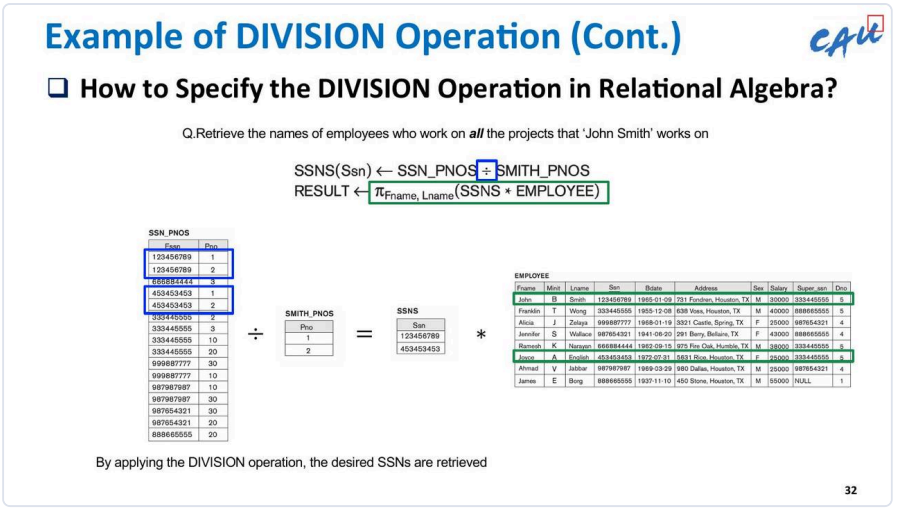
- Form: **R(Z) ÷ S(X) = T(Y)**, where X ⊆ Z (the attributes of S are a subset of those of R) and **Y = Z – X**
- T contains only the Y values that **appear in R in combination with every value in S** (the denominator S restricts the numerator R)
- e.g., R(A, B) = {(a1,b1), (a2,b1), (a3,b1), (a4,b1), (a1,b2), (a3,b2), (a2,b3), (a3,b3), (a4,b3), (a1,b4), (a2,b4), (a3,b4)}, S(A) = {a1, a2, a3} → T(B) = {b1, b4}

Slide example: names of employees who work on **all** the projects that 'John Smith' works on

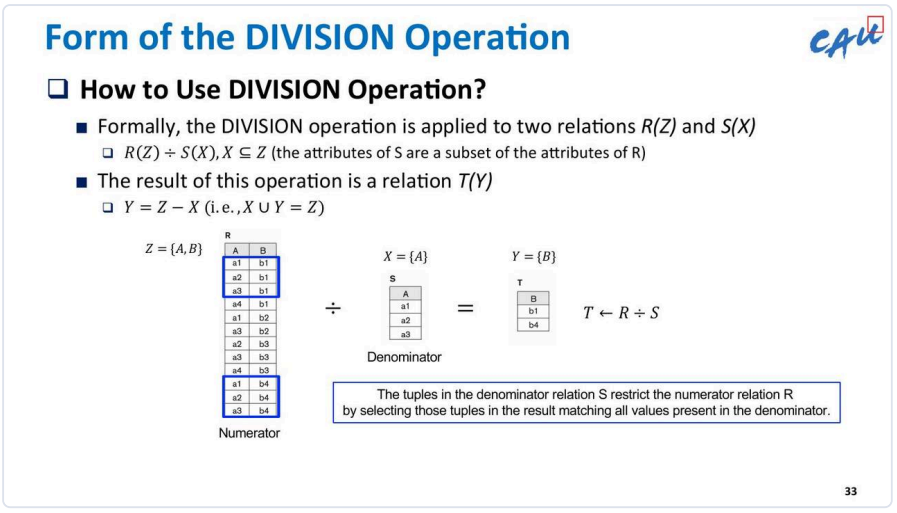
```
SMITH      ← σ Fname='John' AND Lname='Smith' (EMPLOYEE)
SMITH_PNOS ← π Pno (WORKS_ON ⋈ Essn=Ssn SMITH)            -- {1, 2}
SSN_PNOS   ← π Essn, Pno (WORKS_ON)
SSNS(Ssn)  ← SSN_PNOS ÷ SMITH_PNOS                        -- {123456789, 453453453}
RESULT     ← π Fname, Lname (SSNS * EMPLOYEE)
```

→ John Smith, Joyce English

> Exam point: the attribute order and names of the numerator R must line up. SSN_PNOS(Essn, Pno) ÷ SMITH_PNOS(Pno) → the remaining attribute is Essn. It was renamed to SSNS(Ssn) so the result can be natural-joined with EMPLOYEE.



06 p.32 · SSN_PNOS ÷ SMITH_PNOS



06 p.33 · R(Z) ÷ S(X) = T(Y), Y = Z − X

§ Complete set of relational algebra operations

The six operations **{σ, π, ∪, ρ, −, ×}** form a complete set. Every other (original) relational algebra operation can be expressed as a combination of these six.

- **Intersection**: R ∩ S ≡ (R ∪ S) − ((R − S) ∪ (S − R))
- **Join**: R ⋈<condition> S ≡ σ<condition>(R × S)
- **Natural join**: RENAME before a Cartesian product, then SELECT and PROJECT after it
- Strictly, ∩ and ⋈ are not needed, but they are kept as separate operations **because they are used so often**

Complete Set of Relational Algebra Operations

❑ A Complete Set of Relational Algebra Operations

- The following six operations are elements of this set

$$\{\sigma, \pi, \cup, \rho, -, \times\}$$

- ❑ Any other original relational algebra operations can be expressed as a sequence of these operations

INTERSECTION

$$R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$$

JOIN

$$R \bowtie_{<\text{condition}>} S \equiv \sigma_{<\text{condition}>}(R \times S)$$

JOIN = **CARTESIAN PRODUCT** followed by **SELECT**

Strictly, **INTERSECTION** and **JOIN** are not required.

However, they are important to include as separate operations since they are very commonly used in DB applications

NATURAL JOIN can be specified as a **CARTESIAN PRODUCT**, preceded by **RENAME** and followed by **SELECT** and **PROJECT**

34

06 p.34 · Complete set {σ, π, ∪, ρ, −, ×}

Takeaways: Relational Algebra Operations

	OPERATION	PURPOSE	NOTATION
Unary Relational Operations	SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{<\text{selection condition}>}(R)$
	PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{<\text{attribute list}>}(R)$
Binary Relational Operations	THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{<\text{join condition}>} R_2$
	EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{<\text{join condition}>} R_2$, OR $R_1 \bowtie_{(<\text{join attributes 1}>), (<\text{join attributes 2}>)} R_2$
	NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{<\text{join condition}>} R_2$, OR $R_1 *_{(<\text{join attributes 1}>), (<\text{join attributes 2}>)} R_2$ OR $R_1 * R_2$
Set Operations	UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
	INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
	DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
	CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$

44

06 p.44 · Summary table of relational algebra operations

Additional Operations: Aggregation, Grouping, Outer Joins

Slides: 6-1 p.36–43

§ Aggregate functions and grouping (ℑ)

The basic operations cannot express **aggregates (SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT)**, **aggregates per group**, or **outer joins** → additional operations **enhance the expressive power**.

<grouping attributes> ℑ <function list> (R) (ℑ = script F)

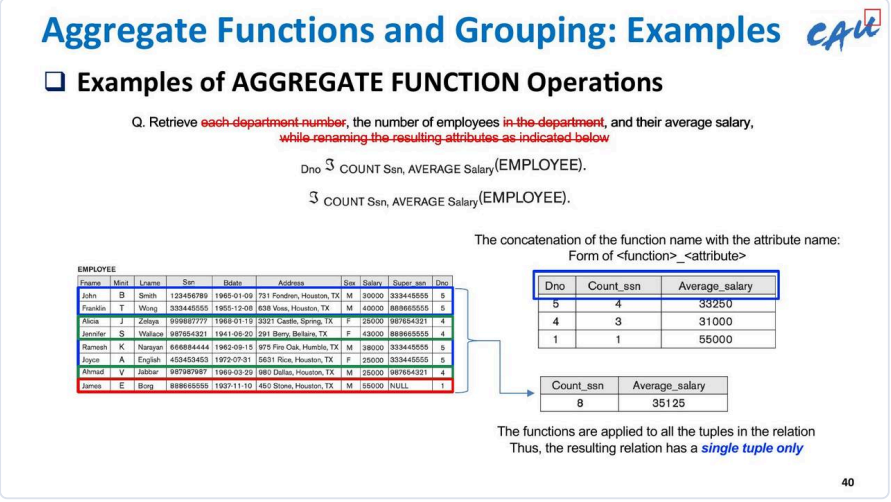
- Grouping attributes: attributes of R used to form groups (may be omitted)
- Function list: pairs (<function> <attribute>). Function = SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT
- **Result**: the grouping attributes + one attribute per element of the function list

Example

```
ρ R(Dno, No_of_employees, Average_sal) ( Dno ℑ COUNT Ssn, AVERAGE Salary (EMPLOYEE) )
```

→ (5, 4, 33250), (4, 3, 31000), (1, 1, 55000)

- **Without renaming**: attribute names = `<function>_<attribute>` → Dno, Count_ssn, Average_salary
- **Without grouping attributes**: the functions apply to **all tuples** → the result is **a single tuple**: `ℑ COUNT Ssn, AVERAGE Salary (EMPLOYEE)` → (8, 35125)



06 p.40 · Result without renaming (Count_ssn, Average_salary) and the case with no grouping attributes

§ Properties of aggregation

- **Duplicates are not eliminated** when an aggregate function is applied (so that sums and averages of salaries come out right)
- **NULL values are not considered** in aggregation
- Relational algebra is a **closed mathematical system**: an aggregate result is also a **relation**, not a scalar number (even with one value in one tuple)

§ Outer joins

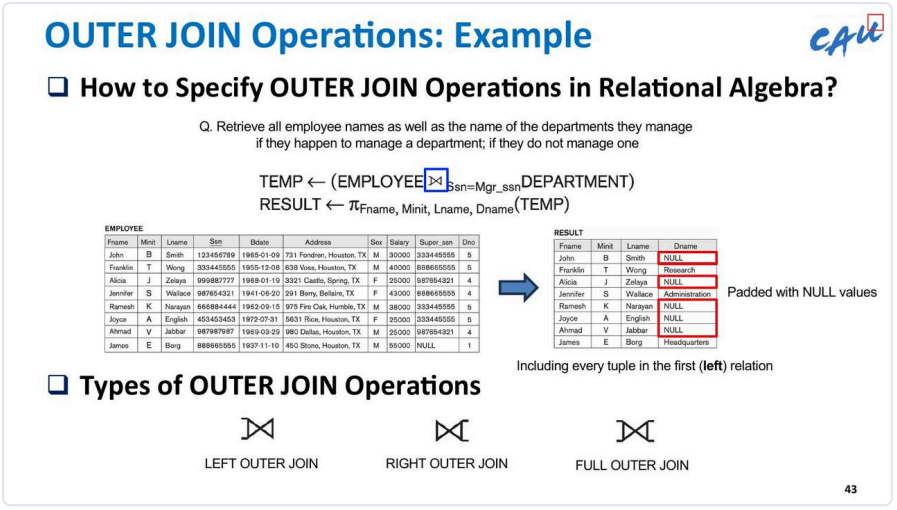
EQUIJOIN and NATURAL JOIN (inner joins) keep **only tuples that satisfy the condition** and drop tuples without a match. Use an outer join **when you want to keep all tuples of one (or both) relations** even without a match.

- e.g., all employee names, plus the name of the department each one manages if they manage one (NULL if not)

```
TEMP   ← EMPLOYEE ⟕ Ssn=Mgr_ssn DEPARTMENT
RESULT ← π Fname, Minit, Lname, Dname (TEMP)
```

→ 8 rows. Wong–Research, Wallace–Administration, Borg–Headquarters; the other 5 have Dname = NULL

- Kinds: **LEFT OUTER JOIN** ⟕ (all tuples on the left), **RIGHT OUTER JOIN** ⟖ (all tuples on the right), **FULL OUTER JOIN** ⟗ (both). Attributes of the unmatched side are **padded with NULL**



06 p.43 · EMPLOYEE ⟕ Ssn=Mgr_ssn DEPARTMENT

Relational Algebra Writing Practice

Slides: db-exercises Week 7 (p.55–59)

§ Exercises 1–2: join paths

Exercise 1: the name and address of all employees who work for the 'Research' department

```
RESEARCH_DEPT ← σ Dname='Research' (DEPARTMENT)
RESEARCH_EMPS ← RESEARCH_DEPT ⋈ Dnumber=Dno EMPLOYEE
RESULT        ← π Fname, Lname, Address (RESEARCH_EMPS)
```

Exercise 2: for every project located in 'Stafford', the project number, the controlling department number, and the manager's last name, address and birth date

```
STAFFORD_PROJS ← σ Plocation='Stafford' (PROJECT)
CONTR_DEPTS    ← STAFFORD_PROJS ⋈ Dnum=Dnumber DEPARTMENT
PROJ_DEPT_MGRS ← CONTR_DEPTS ⋈ Mgr_ssn=Ssn EMPLOYEE
RESULT         ← π Pnumber, Dnum, Lname, Address, Bdate (PROJ_DEPT_MGRS)
```

→ (10, 4, Wallace, 291 Berry, Bellaire, TX, 1941-06-20), (30, 4, Wallace, ...)

Tip: apply σ **first** to the relation that has the condition → follow FK–PK links with ⋈ → finish with π.

§ Exercise 3: union

Project numbers of projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project

```
SMITHS(Essn)        ← π Ssn (σ Lname='Smith' (EMPLOYEE))
SMITH_WORKER_PROJS  ← π Pno (WORKS_ON * SMITHS)
MGRS                ← π Lname, Dnumber (EMPLOYEE ⋈ Ssn=Mgr_ssn DEPARTMENT)
SMITH_MANAGED_DEPTS(Dnum) ← π Dnumber (σ Lname='Smith' (MGRS))
SMITH_MGR_PROJS(Pno)      ← π Pnumber (SMITH_MANAGED_DEPTS * PROJECT)
RESULT              ← SMITH_WORKER_PROJS ∪ SMITH_MGR_PROJS
```

- The two results' attribute names are **both made Pno** so ∪ can be applied (type compatibility)
- Renaming to SMITHS(Essn) makes the **natural join** with WORKS_ON match on Essn

§ Exercises 4–5: difference and intersection

Exercise 4: names of employees who have **no** dependents

```
ALL_EMPS             ← π Ssn (EMPLOYEE)
EMPS_WITH_DEPS(Ssn)  ← π Essn (DEPENDENT)
EMPS_WITHOUT_DEPS    ← ALL_EMPS - EMPS_WITH_DEPS
RESULT               ← π Lname, Fname (EMPS_WITHOUT_DEPS * EMPLOYEE)
```

Exercise 5: names of **managers who have at least one dependent**

```
MGRS(Ssn)            ← π Mgr_ssn (DEPARTMENT)
EMPS_WITH_DEPS(Ssn)  ← π Essn (DEPENDENT)
MGRS_WITH_DEPS       ← MGRS ∩ EMPS_WITH_DEPS
RESULT               ← π Lname, Fname (MGRS_WITH_DEPS * EMPLOYEE)
```

→ Wong, Wallace

> Exam point: "has no …" = **difference (–)**, "A and also B" = **intersection (∩)**, "A or B" = **union (∪)**, "all …" = **division (÷)**. Memorize them as one-to-one counterparts of SQL's NOT EXISTS / EXISTS AND EXISTS / UNION / NOT EXISTS (EXCEPT).